HE-OFT: Privacy-Preserving One-Shot Federated Fine-Tuning of Pretrained Models under Homomorphic Encryption

Halil İbrahim Kanpak, Sinem Sav, Alptekin Küpçü

***Abstract*—Organizations adapt pretrained models to private data by fine-tuning, and parties holding complementary data over the same task would each gain from a jointly adapted model, yet cannot pool their data to build one. Federated learning offers the collaboration but concentrates its privacy risk at training time: gradients and per-round updates permit reconstruction of training data and support membership inference substantially stronger than any attack on the finished model. A protocol that exchanges a single encrypted contribution removes that surface, but it still ends by handing the finished model to every participant, which is not permitted where the model is itself a regulated or proprietary asset. We present HE-OFT, a federated fine-tuning protocol in which the model is never disclosed to any party. Each client fine-tunes a low-rank adapter and a classifier head on a frozen public backbone, retains the adapter locally, and uploads one encrypted head displacement; the server combines these under multiparty CKKS and never decrypts the result. Queries are answered under encryption, and a quorum of clients returns only the predicted label, addressed to the party that asked. Withholding the model determines the design, since the querying party must form its query from public components alone, so the shared trained quantity cannot reside inside the backbone. Because the server cannot read its inputs, it cannot select an aggregation rule, and we give an estimator that lets the federation choose between two servable arrangements while disclosing only the index of the one selected; the direct procedure, in which each client scores both on its own held-out data, estimates the wrong quantity and selects incorrectly in every case we measured. Across four text classification tasks and one vision task, the protocol yields a model substantially stronger than any client obtains alone, at a cost of 0.03 to 0.14 accuracy relative to a disclosed model. A real multiparty CKKS implementation reproduces the plaintext computation within the approximation bounds of the scheme, so the cryptographic layer costs no accuracy, and the recurring communication is a few megabytes per query.**

***Index Terms*—Federated learning, one-shot federated learning, federated fine-tuning, homomorphic encryption, multiparty computation, privacy-preserving machine learning.**

I. Introduction

Organizations adapt public pretrained models to their own data by fine-tuning [1], [2]. Often several organizations hold complementary data over the same task, different patient populations, fraud patterns, or customer bases, and each would obtain a better model from the combination than from its own data alone. They cannot pool that data. Data-protection law restricts the sharing of confidential records across institutions, imposing data-minimisation and cross-border-transfer limits and, for health data, a de-identification bar before any disclosure, under regimes from the EU General Data Protection Regulation and the U.S. Health Insurance Portability and Accountability Act to Turkey's Personal Data Protection Law and a widening set elsewhere [3], [4], [5], [6], [7]. Competitive concerns point the same way, and handing the data to whoever operates the fine-tuning infrastructure raises the same objection [8].

Federated learning is the standard response, and its privacy risk is concentrated at training time rather than in the model it produces. The protocol exchanges model updates instead of data, but the updates are themselves the vulnerability. A gradient permits reconstruction of the batch that produced it [9], [10]. An observer of the per-round updates mounts membership inference far stronger than anything possible against the finished model: on the same model and dataset, an attack that reaches 87% accuracy against the update stream falls to 54.5% against the final model alone [11]. A server free to choose the weights it broadcasts can harvest client inputs outright [12], [13]. The decisive quantities are therefore those a protocol exposes while the model is being built, and the number of times it exposes them.

Two measures remove that surface together. Making the protocol *one-shot*, so that each client contributes once, means there is no intermediate state for anyone to observe. Encrypting that single contribution means the one thing that is exchanged is never available in plaintext. We use one-shot in this sense throughout: not as a claim that the parties never communicate again, but as the statement that no intermediate training artifact is ever exposed. Prior work applies one of these measures at a time. One-shot federated learning sends its single contribution in plaintext, or protects it with differential privacy, which noises the very artifact it releases. Encrypted federated learning runs many rounds of encrypted training or encrypted aggregation, paying the cryptographic cost on every round, and the direct route of training under encryption forces polynomial replacements of every activation and a deep, repeatedly bootstrapped circuit [14].

Applying both still leaves one artifact exposed, namely the model itself. Every protocol above ends by handing the finished model to the participants, which is reasonable when

Corresponding Author: Sinem Sav (sinem.sav@cs.bilkent.edu.tr)

Halil İbrahim Kanpak is with the Department of Computer Engineering, Koç University, İstanbul, Türkiye (e-mail: hkanpak21@ku.edu.tr).

Sinem Sav is with the Department of Computer Engineering, Bilkent University, Ankara, Türkiye (e-mail: sinem.sav@cs.bilkent.edu.tr).

Alptekin Küpçü is with the Department of Computer Engineering, Koç University, İstanbul, Türkiye (e-mail: akupcu@ku.edu.tr).

Manuscript received XX XX, XXXX; revised XX XX, XXXX.

the purpose is to distribute a shared model, and unacceptable when the model may not be distributed at all. A model fine-tuned on confidential records can itself be a regulated artifact, and in high-risk domains such as health, biometrics and credit it is treated as one [15]. It can also be the asset the participants are least willing to give each other. For such deployments the relevant question is not how to release a model safely, but how to construct one that is never released.

This paper presents such a protocol. The parties fine-tune on a frozen public backbone, contribute once under multiparty CKKS, and obtain a shared classifier that no party ever decrypts. It answers queries under encryption and returns a label, so a participant learns what the model predicts on its own inputs and nothing else about the model. There is no round in which a training artifact is exposed, no plaintext contribution, and no plaintext model at any point in the protocol's lifetime.

Withholding the model is not simply a stronger form of withholding the updates. It changes what may be shared at all. A model that is never decrypted must still answer questions, and the party asking a question has to form it from components it can run itself, which means those components must be public. The shared trained quantity therefore cannot sit inside the backbone, since the features would then depend on it and computing them would require evaluating every nonlinearity of the network homomorphically. It has to attach after the last nonlinearity, and in a transformer there is exactly one such place. This single requirement determines the design, and the paper's three contributions are its consequences.

- **Partitioning the trainable unit.** Each client keeps its own adapter, which is trained locally and never transmitted, and only the classifier head is federated. The division is one of coverage: a client can improve its own representation alone, but it cannot learn to recognise a class it has never observed, and that is what the federation supplies. Because no aggregate of the adapters is ever formed, a coalition of all but one client has no sum from which to subtract its own contributions (section III-D).
- **Inference on an encrypted model.** The querying client computes its own features and encrypts them, so the serving party cannot invert them to recover the query. The serving party applies the encrypted head, reduces the logits to the index of the largest entry under encryption, and a quorum switches that index to the querier's key so that no other client learns the answer. Returning only the label denies the linear solve that logits would permit (section III-E).
- **Selection under encryption.** Two arrangements are servable at the same cost, and which is preferable depends on the task. The direct procedure, in which each client measures both on its own held-out data and votes, estimates the wrong quantity and selects incorrectly whenever the two differ. We give an estimator that corrects it, runs under encryption, has no fitted threshold, and reveals only which arrangement won (section III-F).

We evaluate on four text classification tasks and one vision task, with frozen RoBERTa and ViT backbones, and we implement the cryptographic protocol in real multiparty CKKS rather than simulating it. We report what never decrypting the model costs, in accuracy against a released model and in the communication and computation the protocol adds, including the traffic that recurs per query, which a claim of one-shot operation might otherwise obscure.

Each of these claims rests on a premise concerning where federated learning leaks, and on what existing work already provides. Section II establishes both.

II. Background and Related Work

Five lines of work bear on this paper: the leakage properties of federated learning, the cryptographic tools that protect training, one-shot federated learning and its differentially private variants, federated fine-tuning of adapters, and secure inference. We take them in that order, since the first establishes the premise on which the design rests and the remainder determine its position. The closing paragraph returns to the claim of section I and identifies what existing work leaves unaddressed.

A. Leakage at Training Time and at Inference Time

Federated learning began as an iterative procedure in which a server repeatedly broadcasts a model, clients improve it on their own data, and the server averages the returned updates [16]. Every round of that loop reveals a fresh quantity computed on private data, and a decade of attacks has established that these training-time quantities constitute the most damaging disclosure such a protocol makes.

A gradient permits reconstruction of the batch that produced it, pixel-accurate on images [9] and at high resolution even from trained networks, where averaging over a hundred images or a hundred local steps still leaves individual inputs recoverable [10]. An observer of the per-round updates infers membership and unintended properties of another client's data: with the update stream of a federated run on CIFAR-100 a passive server reaches 79.2% membership accuracy and an active one 87.3%, against 54.5% for a black-box attack on the same finished model, and the source identifies the repeated per-round updates as the factor that boosts the attack [11]. Accuracy grows with the number of rounds observed [17], and update-level membership inference has reached 0.99 precision at perfect recall [18]. A server free to choose the weights it broadcasts does not need inference at all: crafted weights harvest verbatim copies of client inputs, through mini-batches of a hundred points and even through a secure-aggregation layer [12], [13]. Federated distillation, which shares predictions rather than gradients, was hoped to avoid this, but its training-time quantities leak as well: paired-logit inversion recovers images from shared logits [19], and membership can be inferred from distillation outputs and public-set usage [20], [21], [22].

Attacks that see only the finished model are consistently weaker, and the field's central survey draws exactly this line, separating adversaries who observe the intermediate iterates from those who see only the final model [23]. Membership inference against a finished model is a mature literature [24], [25], [26], and against well-generalised models its success

concentrates at low false-positive operating points rather than in bulk [27]. Model inversion needs released confidences and strong priors [28], [29].

The asymmetry admits a straightforward explanation. Training-time quantities are high-dimensional views of one client's data, taken repeatedly during optimisation. The finished model is a single aggregate that generalisation itself pushes away from any individual training point. This asymmetry is the premise of our design: a protocol that never exposes a training-time quantity, and that exchanges anything at all only once, reduces every adversary to the weaker position.

B. Protecting Training with Cryptography

The cryptographic line protects training-time quantities without altering the model, in contrast to differential privacy, which perturbs the quantities themselves and pays for the guarantee in accuracy.

Secure aggregation lets the server learn only the sum of the clients' updates, using masks that cancel in the aggregate [30]. It is lossless, but it protects one round's sum inside an iterative protocol, and the revealed per-round sums have themselves been shown to leak across rounds [31], [32]. Homomorphic encryption goes further by computing on contributions that stay encrypted. In its most ambitious form it trains a network entirely under encryption: POSEIDON runs encrypted stochastic gradient descent under multiparty CKKS, with activations replaced by polynomial approximations so that they can be evaluated homomorphically [14]. The guarantee is strong and adds no noise, but the approach is iterative and pays for encrypted nonlinearity with a deep multiplicative circuit, repeated bootstrapping, and hours of wall-clock time. The delicacy of polynomial activations is documented by a substantial line of work [33], [34], [35], [36], [37].

A lighter line encrypts only the aggregation of an otherwise plaintext training loop. BatchCrypt, FedML-HE and FedSHE develop packed and quantised encrypted summation for cross-silo training [38], [39], [40], SHE-LoRA selectively encrypts a sensitivity-chosen subset of a low-rank adapter each round [41], and hybrid multi-key schemes reduce a round of secure summation to one client-to-server transmission [42]. All of these remain bound to the iterative protocol: the encryption cost is paid on every round, and what is protected is a per-round sum rather than a complete contribution. SHE-LoRA is also instructive on key structure. It uses single-key homomorphic encryption and therefore has to assume that the server and the clients do not collude, since any client that revealed its key to the server would undo the protection. The threshold structure of section III-C discharges that assumption rather than imposing it.

Transfer learning itself has been placed under encryption. HETAL trains a softmax classifier head on frozen features entirely under CKKS [43], following a longer line that runs linear and logistic training homomorphically [44], [45], [46]. A federated variant encrypts the fine-tuned last layer across many rounds [47], a concurrent scheme averages encrypted feature tokens under a single decryption key [48], and recent schemes run low-rank-adapter fine-tuning under encryption between a model owner and data owners [49], [50]. These share our structural niche, a frozen public backbone with a small trainable unit, but they run an optimiser on ciphertexts, so multiplicative depth grows with the step count and every nonlinearity must be approximated. Most are also single-party, one data owner outsourcing computation rather than a collaboration of mutually distrustful parties. They solve a different problem: outsourcing training for a party that cannot train locally, where our parties can train locally but must not reveal what they trained.

C. One-Shot Federated Learning and Differential Privacy

A parallel line collapses the many rounds into a single exchange [51], [52]. Early knowledge-transfer methods had clients exchange predictions on a shared public set [53], [54], in practice repeating the exchange, and ensemble distillation trains a server model against aggregated client predictions over many rounds [55]. Genuinely single-round methods arrived with data-free distillation, in which the server distils the clients' models through a generator or synthesised inputs [56], [57], and with fusion methods that stitch client models together [58] or aggregate them through a layer-wise posterior [59], [60]. These establish that one round can produce a usable model, but they operate in plaintext: the single contribution each client sends, an entire model or a synthetic distillate of its data, is exposed, and by the evidence of section II-A that is precisely the quantity worth attacking.

The one-shot methods that do address privacy use differential privacy. FedAUXfdp releases client contributions under a differentially private mechanism after distillation through a pretrained extractor [61], and related approaches protect a diffusion-generated [62] or teacher-ensemble [63] surrogate, or add noise-free differential privacy to the distillation itself [64]. These are our natural quantitative peers, and their privacy is lossy: the noise a meaningful budget requires is added to the very artifact that is released, so accuracy falls as the budget tightens. The guarantee also differs in kind. Differential privacy proves a statistical bound on how much a released artifact can reveal and buys that bound with accuracy. Encryption makes a contribution computationally indistinguishable from random to every party without the key and costs the model nothing. The same trade-off structures differentially private transfer learning, where noisy gradient descent on frozen features reaches strong accuracy at moderate budgets [65], [66], [67], [68], extended to federated low-rank adaptation [69], [70]. Those methods share our backbone-and-adapter structure and confirm that frozen public features are a good substrate for private learning, but they pay a budget-dependent accuracy tax on the model they release, and they are centralised or multi-round.

D. Federated Fine-Tuning and Task Arithmetic

A recent line federates parameter-efficient fine-tuning, communicating a low-rank adapter rather than a model. FedIT averages the adapters round by round [71], but per-factor averaging is biased: a client learns a product of two factors, and averaging the factors separately does not give the

average of the products. FLoRA removes the discrepancy by stacking factors [72], FlexLoRA by reconstructing full products and re-factoring [73], HetLoRA by reconciling heterogeneous ranks [74], and FedSA-LoRA by sharing only one factor [75]. FFA-LoRA freezes the randomly initialised down-projection and trains only the up-projection, so that averaging the trained factor is exact; it was introduced to stabilise multi-round differentially private tuning [76]. Algebraically, combining displacements from a shared initialiser is task-vector merging [77], which has been shown equivalent to one-shot federated averaging [78], and that one round of fine-tuning suffices for foundation models is established in plaintext [79]. What none of this line provides is any protection of the contributions, which is our subject.

E. Secure Inference

Answering queries from a model that stays encrypted is the subject of secure inference. Non-interactive systems evaluate a transformer under CKKS on GPUs [80], and interactive protocols combine homomorphic encryption with secret sharing to evaluate transformers between two or three parties [81], [82], [83], with the nonlinearities of attention as the dominant cost. This literature is what makes the disclosure setting of section III-E affordable, and we compose with it rather than re-solve it: the construction here serves a trained quantity that is linear in the features, and serving a trained quantity that sits inside the backbone reduces to exactly the problem these systems address. One difference should be stated. These systems evaluate a plaintext model on encrypted inputs, whereas the shared weights here are encrypted as well, so each site would carry a ciphertext-by-ciphertext product rather than a ciphertext-by-plaintext one. Since the dominant cost in that literature is the nonlinearities, which are unchanged, the difference is an additive term rather than a change of order.

Positioning. The evidence of section II-A says that the decisive attack surface is the one exposed while training runs, and the four lines that follow each remove part of it. The multi-round cryptographic line protects every round but pays on every round, and what it protects is a per-round sum. The one-shot line exposes its single contribution in plaintext, or noises it and pays in accuracy. The fine-tuning line communicates a small object but protects nothing. Every one of them, moreover, ends by handing the finished model to the participants, so the model itself remains a disclosed artifact. A single prior scheme is at once one-shot, federated and encrypted, but only for a single-layer closed-form learner that cannot adapt a deep representation [84].

What remains unaddressed is a protocol that exposes no training-time quantity, that exchanges anything at all only once, and that never discloses the model even to the parties that built it. The third of these is not a refinement of the first two. It changes what can be shared, because a model that is never decrypted must still answer questions, and the party asking must be able to form a query from public components alone. Section III works out what that forces, and the rest of the paper measures what it costs.

III. Method

A. Setting

We consider N clients holding private labelled data over a common label space of C classes. Client j holds $\mathcal{D}_j$ of size n_j, and the data are heterogeneous: each client's class proportions are drawn from a Dirichlet distribution of concentration α, so a smaller α leaves each client a few dominant classes and almost none of the rest. The clients want a classifier that works across the whole label space, and they cannot pool their data to build one.

Three requirements separate this setting from ordinary federated learning. The protocol is *one-shot*: each client uploads once, and there is no iterative exchange. Privacy is *cryptographic*: a client's contribution is never available in plaintext to any other party, and we do not accept the accuracy loss that differential privacy incurs when it is the only protection. And the model is *never disclosed*: no party, client or server, ever holds the trained classifier in plaintext. The third requirement is the one that shapes everything below, and section III-G states precisely what it buys.

Each requirement forecloses a family of designs, and the remaining space is narrow. We record the chain of implications here, so that the choices made in the rest of this section can be read as consequences of the requirements rather than as independent design preferences.

- **C1** *Optimisation cannot run under encryption at acceptable cost.* Gradient steps, softmax, and adaptive curvature require deep, repeatedly bootstrapped homomorphic circuits. *Therefore* all learning happens in plaintext on the clients, and the server performs no learning.
- **C2** *A server computing on ciphertexts cannot read its inputs.* It cannot inspect, compare, or branch, so it cannot select a data-dependent aggregation rule. *Therefore* the choice is deferred to the clients, who make it without decrypting anything (section III-F).
- **C3** *Independently trained models do not combine linearly.* In the parameter space of a deep network, models trained from different starting points occupy incomparable regions. *Therefore* every client starts its trainable unit from one public initialiser on one frozen public backbone.
- **C4** *Multiplicative depth dominates homomorphic cost. Therefore* the aggregation is a linear combination, and the design is arranged so that this cheapest operation is exactly the one the method needs.
- **C5** *Intermediate training signals are the strongest attack surface.* Each exposed round is a fresh training-time artifact, far more revealing than a finished model (section II-A). *Therefore* the protocol is one-shot, and its single artifact is a ciphertext.
- **C6** *No single party may be able to decrypt.* The parties are mutually distrustful and the server is untrusted. *Therefore* the secret key is generated and held distributively, under multiparty CKKS (section III-C).
- **C7** *The model is never disclosed, yet it must answer queries.* A query is formed from features, and the client that asks must be able to compute those features itself. *Therefore* everything the client runs is public, and the trained

TABLE I
NOTATION.

Symbol	Meaning
N	number of participating clients
$\mathcal{D}_j,\ n_j$	client j's dataset and its size $n_j = \lvert\mathcal{D}_j\rvert$
C	number of classes
$n_{j,c}$	examples of class c held by client j
α	Dirichlet concentration, the label-skew parameter
φ	frozen public backbone, never trained or encrypted
A_j	client j's adapter, trained locally and never transmitted
φ_j	client j's feature map, the backbone composed with A_j
θ_0	public initialiser of the classifier head
h_j	client j's trained head
Δ_j	head displacement $h_j - \theta_0$
w_j	sample weight $n_j / \sum_i n_i$
$\theta^\star$	shared head, held only as a ciphertext
$\mathsf{Enc}(\cdot)$	encryption under the collective public key

quantity that is shared cannot sit inside the backbone. This requirement distinguishes the present design from one in which the model is released, and section III-D derives its consequences.

Constraints **C1** through **C6** are shared with any one-shot encrypted federation. **C7** is particular to never disclosing the model, and the three contributions of this paper follow from it: a partition of the trainable unit (section III-D), a procedure for answering queries without decrypting the model (section III-E), and a procedure by which the federation selects between candidate arrangements it cannot observe (section III-F).

B. Notation

Table I collects the symbols. Client indices are $j \in \{1, \dots, N\}$ and class indices are $c \in \{1, \dots, C\}$.

C. Multiparty Homomorphic Encryption

We build on CKKS [85], a ring-learning-with-errors construction [86] that performs approximate arithmetic on vectors of real numbers under encryption. CKKS packs many real slots into one ciphertext and supports addition of ciphertexts, multiplication of a ciphertext by a plaintext, and multiplication of two ciphertexts. It is a levelled scheme, so each multiplication consumes one level of a finite budget, and a depleted budget is restored by bootstrapping [87].

A single-key scheme would require one party to hold the secret key and therefore to be trusted with decryption. We use the multiparty variant of CKKS [88], in which the secret key is shared among the clients through a distributed key-generation protocol that produces a collective public key without ever assembling the secret in one place. Any party, the server included, may compute on ciphertexts under the collective public key, but decryption requires the cooperation of a quorum of clients and is carried out as a collective key-switching protocol. Two further protocols of the same family are used below: a key switch to a designated public key, which re-encrypts a result so that one chosen party can read it, and a collective refresh, which restores a depleted level budget and plays the role bootstrapping plays in the single-key setting.

We state the reason for this choice. The aggregation of section III-D uses only additions and multiplications by public scalars, so it consumes one level. What we need from the scheme is not depth but key structure: threshold decryption over a collectively generated key is the property the protocol cannot do without, and mature distributed key generation and collective key switching provide it directly. CKKS additionally packs thousands of real values into one ciphertext and multiplies them by public scalars natively. Additively homomorphic schemes offer neither packed real arithmetic at this width nor comparable multiparty tooling, and masking-based secure aggregation reveals the plaintext sum to the server, which this protocol never does.

D. Partitioning the Trainable Unit

a) The construction.: Every client uses the same frozen public backbone φ, and trains two things on its own data: a low-rank adapter A_j that adjusts the representation, and a classifier head h_j on top of it. Only the head is shared. Client j encrypts the displacement $\Delta_j = h_j - \theta_0$ of its head from the public initialiser and uploads it once. The server combines the encrypted displacements into a single shared head and never decrypts the result. The adapter is trained locally and never transmitted. Figure 1 shows the arrangement.

b) Necessity of the partition.: The client that asks a question must compute the features of that question itself, by **C7**. If the shared trained quantity were an adapter placed inside the backbone, the features would depend on it, and computing them would require evaluating the backbone with encrypted weights. Every nonlinearity of the network would then have to be evaluated homomorphically, which is the cost this design exists to avoid. The trained quantity that is shared must therefore attach after the last nonlinearity, and in a transformer the only such point is the final linear map. That is the head.

c) What the argument fixes, and what it leaves open.: The requirement constrains the *position* of the shared map, not the task it serves. Any trained linear map that sits after the last nonlinearity satisfies it, provided the querying client can compute the features that enter the map. A classifier head is one such map. The vocabulary projection of an autoregressive decoder is another, since it is linear, sits after the final layer normalisation, and takes features the client can compute from a prefix it generated itself. The construction therefore does not depend on the label space being a set of classes. We evaluate classification only, and section V-H records what a generation setting would additionally require, including the restriction to greedy decoding that section III-E imposes.

The adapter is relocated rather than discarded. Each client retains its own, so the representation still adapts to local data, and the federation shares the part where sharing is necessary. The distinction follows from coverage: a client can improve its own representation alone, but it cannot learn to recognise a class it has never observed. Coverage is a property of the head, which is exactly what the federation supplies. Section V-B measures both halves of this claim.

Retaining the adapter locally has a second consequence, which is cryptographic. No aggregate of the adapters is

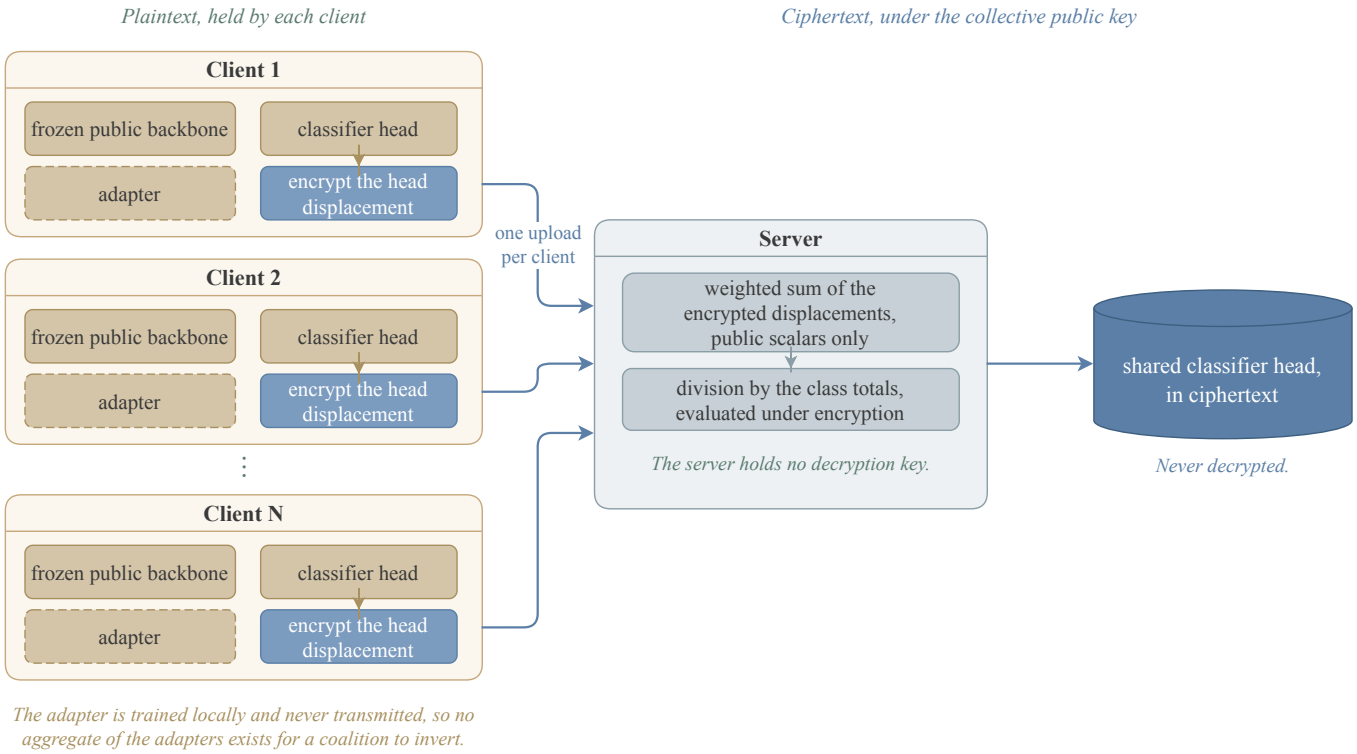

Fig. 1. Building the shared head. Every client fine-tunes on the same frozen public backbone. The adapter is trained locally and never transmitted, so no aggregate of the adapters is ever formed and a coalition of clients has no sum from which to subtract its own contributions. Only the head displacement is encrypted and uploaded. The server adds the ciphertexts, weighting each by a public count, and divides by the per-class totals under encryption, holding no decryption key. The result is a shared head that no party decrypts.

ever formed. A coalition of $N-1$ clients therefore has no adapter sum from which to subtract its own contributions, and cannot recover the remaining client's adapter by arithmetic. Section III-G states this precisely.

d) The aggregation.: Let $g_j \in \mathbb{R}^C$ collect client j's per-class example counts, $g_{j,c} = n_{j,c}$. The shared head is the coverage-weighted combination

$$\theta^\star = \theta_0 + \frac{\sum_{j=1}^{N} g_j \odot \Delta_j}{\sum_{j=1}^{N} g_j}, \tag{1}$$

in which row c of the shared head is the count-weighted average of the clients' row-c displacements. Clients holding a class decide its row, and rows that no client covers remain at the public initialiser. Each client forms the products $g_j \odot \Delta_j$ locally, before encryption, so the server only adds ciphertexts.

e) The division.: The denominator of eq. (1) is the vector of federation-wide per-class totals. It cannot be decrypted. Every client knows its own counts, so a coalition of $N-1$ clients that learned the totals would subtract its own and read the remaining client's class histogram exactly, which for an institutional participant is a description of its case mix. The reciprocal is therefore evaluated under encryption, using an inversion circuit over the positive domain whose refreshes are collective. This is the one deep operation in the construction of the shared head, and it is paid once, at aggregation, never per query. Section V-E reports its measured cost.

f) Cost.: The server performs one ciphertext addition per client per ciphertext, one encrypted reciprocal, and one multiplication. The encrypted object is the head alone, whose size is set by the number of classes and the feature dimension and not by the backbone behind it. Communication is one upload per client and no download of model parameters at all, since no client ever receives the shared head.

Algorithm 1 Building the shared head. All clients hold the same frozen public backbone φ and the same public head initialiser θ_0.

1: clients run distributed key generation, obtaining a collective public key
2: **for** client $j = 1, \ldots, N$ **in parallel do**
3: train adapter A_j and head h_j on $\mathcal{D}_j$ ▷ plaintext, local
4: **keep** A_j; it is never transmitted
5: send $\mathsf{Enc}(g_j \odot \Delta_j)$ and $\mathsf{Enc}(g_j)$, where $\Delta_j = h_j - \theta_0$
6: **end for**
7: **server** adds the ciphertexts to form the numerator and the denominator of eq. (1) ▷ additions only
8: **server** evaluates the reciprocal of the denominator under encryption and multiplies ▷ once, not per query
9: **return** $\mathsf{Enc}(\theta^\star)$, which no party decrypts

E. Inference on the Encrypted Head

a) The construction.: A client that wants a prediction computes the features of its query with its own backbone and adapter, encrypts them, and sends the ciphertext to a serving party. The serving party applies the encrypted shared head to the encrypted features, obtains encrypted logits, and reduces

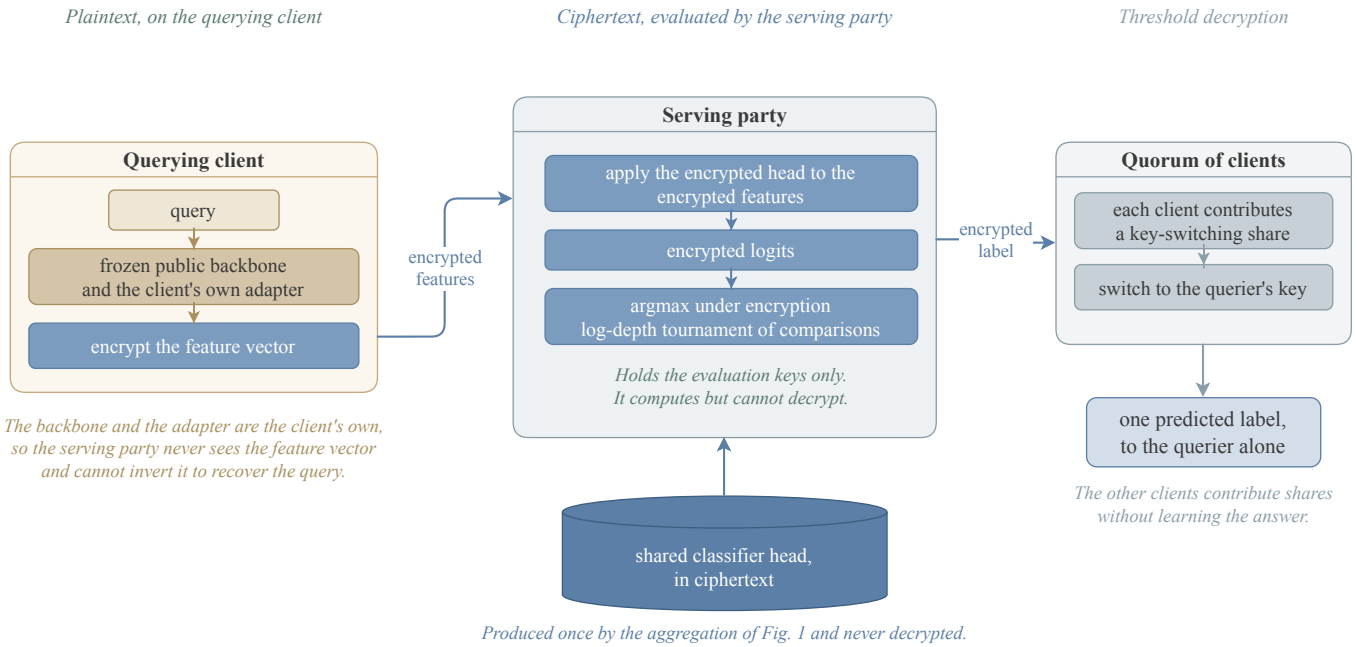

Fig. 2. Answering one query. The querying client computes the features with its own backbone and adapter and encrypts them, so the serving party never sees the feature vector and cannot invert it to recover the query. The serving party holds the evaluation keys only: it applies the encrypted head, obtains encrypted logits, and reduces them to the index of the largest entry by a tournament of homomorphic comparisons. A quorum of clients then switches that index to the querying client's key, so the other clients contribute shares without learning the answer.

them to the index of the largest entry under encryption. A quorum of clients then key-switches that index to the querying client, which decrypts one label. Figure 2 shows the path, and algorithm 2 states it.

b) Encryption of the query.: The serving party could be given the features in plaintext, which would make the head application cheaper. We encrypt them because features are invertible: a party holding $\varphi_j(x)$ and the public backbone can reconstruct an approximation of x. Sending features in plaintext would therefore hand the serving party the client's input, which is the thing the protocol exists to protect. The price is that the head is applied ciphertext by ciphertext rather than ciphertext by plaintext, and section V-E reports what that costs.

c) Restriction to the predicted label.: Returning logits would defeat the construction. The head is a linear map on features the client computes itself, so a client that reads logits for feature vectors of its choosing recovers the head by solving a linear system in as many queries as the feature dimension [89]. This is not a hypothetical concern: the same object, the final projection layer of a production transformer, has been recovered through a deployed interface for a few tens of dollars, using an interface that exposed log-probabilities and a logit bias, and the operators' response was to restrict that combination [90]. Returning only the index of the largest entry is the same restriction imposed by construction rather than by policy. It places an adversary in the hard-label setting, where functionally equivalent extraction is markedly harder and has only recently been achieved for general networks [91], [92]. It does not make extraction impossible, and section III-G states the resulting bound honestly, as a bound on queries rather than a cryptographic hardness claim.

Algorithm 2 Answering one query. The shared head remains a ciphertext throughout.

Require: query x at client j; $\mathsf{Enc}(\theta^\star)$ held by the serving party
1: client j computes $\varphi_j(x)$ with its own backbone and adapter ▷ plaintext, local
2: client j sends $\mathsf{Enc}(\varphi_j(x))$
3: **serving party** computes $\mathsf{Enc}(\ell) \leftarrow \mathsf{Enc}(\theta^\star)\,\mathsf{Enc}(\varphi_j(x))$ ▷ ciphertext by ciphertext
4: **serving party** reduces $\mathsf{Enc}(\ell)$ to $\mathsf{Enc}(\hat{y})$, $\hat{y} = \arg\max_c \ell_c$, by a tournament of homomorphic comparisons [93]
5: a quorum of clients key-switches $\mathsf{Enc}(\hat{y})$ to client j's public key
6: **return** $\hat{y}$, readable by client j alone

d) Directed decryption.: Decryption needs a quorum, so other clients necessarily participate in producing an answer to a question they did not ask. Rather than decrypt the label collectively, which would reveal it to every participant, the quorum key-switches the ciphertext to the querying client's public key. The other clients contribute shares and learn nothing.

e) Cost and the role of the serving party.: The argmax is the one operation in the protocol that is not shallow. Evaluated as a sequential fold of $C-1$ comparisons it costs minutes for a large label space. Packing the logits into one ciphertext and reducing them by a tournament lowers the number of sequential comparisons from $C-1$ to $\lceil \log_2 C \rceil$, and section V-E reports the measured result. None of this is particular to the multiparty setting: homomorphic evaluation under a collectively generated key is identical to evaluation under a single key, and the schemes differ only in key generation and in decryption, so single-key results carry over.

The serving party is not trusted with anything. It holds the collective public key, the evaluation keys, and ciphertexts, so it computes but cannot decrypt, and it learns neither the model nor any client's data. It is trusted only for availability, and where honest computation must be certified, verifiable homomorphic encryption applies [94].

f) Scope.: This construction serves a trained quantity that is linear in the features. Serving a trained quantity that sits inside the backbone is the problem that secure transformer inference addresses, and an encrypted shared adapter composes with any such system at that literature's cost. We do not re-solve it. One difference should be noted: those systems evaluate a plaintext model on encrypted inputs, whereas here the shared weights are encrypted as well, so each adapter site would carry a ciphertext-by-ciphertext product. The dominant cost in that literature is the nonlinearities, which are unchanged, so the added cost is the low-rank product per site rather than a different order of magnitude.

F. Candidate Selection Under Encryption

a) The construction.: Two arrangements are servable at the same cost: the shared head over each client's own adapter, and the shared head over the bare public backbone with no adapter at all. Which is better depends on the task, and the federation must choose without decrypting either. Each client evaluates both on data it holds back from its own training, entirely under encryption, and reports encrypted per-class counts. The counts are combined into one encrypted score per arrangement, the two scores are compared under encryption, and exactly one value is decrypted: which arrangement the federation adopts. Figure 3 shows this, and algorithm 3 states it.

b) Inadequacy of the held-out vote.: The natural procedure is for each client to measure the accuracy of both arrangements on its held-out data and vote. This is not a matter of noise: the procedure estimates the wrong quantity. The arrangement without an adapter is a single model, and the union of the clients' data is the whole training set, so pooled held-out measurements estimate its accuracy on the global distribution closely. The arrangement with personal adapters is N different models, and client j's held-out data can only measure client j's own model. How client j's model performs on the other clients' distributions is not observable locally, and that is precisely where it fails. A client's held-out set also contains only the classes that client holds, so it cannot measure performance on classes the client has never seen. The measurement is systematically optimistic for the personal arrangement, and section V-D shows that the resulting vote chooses wrongly whenever the two differ.

c) A prior-weighted estimator.: Score both arrangements as the expected accuracy of a prediction on an example drawn from the federation's own class distribution,

$$E \;=\; \sum_{c=1}^{C} p_c \, a_c, \qquad p_c \;=\; \frac{\sum_j n_{j,c}}{\sum_j n_j}, \tag{2}$$

where a_c is the accuracy on class c and p_c is that class's share of the federation's examples, taken from the per-class counts already used in eq. (1). Two asymmetries follow from the setting rather than from choice. For the shared arrangement there is one model, so per-class evidence from different clients concerns the same object and combines, and every class is covered by some client. For the personal arrangement each client's evidence concerns only its own model, and classes it does not hold contribute no evidence at all. Scoring those classes at zero makes E a lower bound for the personal arrangement, so the federation adopts it only when even a pessimistic estimate exceeds the accurately measured alternative. There is no fitted threshold anywhere: the rule compares two numbers on the same absolute scale.

Algorithm 3 Choosing an arrangement without decrypting either. Nothing is revealed except the index of the winner.

for each arrangement m and each client j **do**
  for each held-out example (x, y) of client j **do**
    obtain $\mathsf{Enc}(\ell)$ and $\mathsf{Enc}(\max_c \ell_c)$ as in algorithm 2
    select $\mathsf{Enc}(\ell_y)$ with a plaintext mask ▷ the client knows y, so the mask is public
    add $\mathsf{Enc}\big(\mathbf{1}[\ell_y = \max_c \ell_c]\big)$ to the class-y accumulator
  end for
end for
combine the accumulators into $\mathsf{Enc}(E_m)$ by eq. (2), using public scalars ▷ depth one
compare the two encrypted scores
return the decrypted index of the larger ▷ one value

d) Requirements on the held-out split.: The estimator needs an estimate of accuracy on classes a client has not seen, and its only local proxy is accuracy on classes it has barely seen. A held-out set carved uniformly almost never contains a class the client holds one or two examples of, so that proxy is unmeasurable. Each client therefore reserves at least one held-out example from every class it holds before carving the remainder at random. A client holding a single example of a class donates it, so its adapter never trains on that class, which is exactly the probe the estimator needs. This costs a negligible amount of training data and requires no change to the cryptography.

e) Cost and disclosure.: Every step is depth one except the comparisons, which are the same circuit already used for the argmax. The client knows its own labels, so the mask that selects the true class is a public plaintext, and the per-class totals are the same aggregate already used in eq. (1), so the weighting is by public scalars. One value is decrypted for the whole federation. The per-client per-class accuracies are not decrypted, and must not be: together with the count matrix they would describe each client's label distribution alongside its model quality.

G. Threat Model

The participants are N clients, an aggregation server, and a serving party, the last of which may be the server. The server and the serving party are honest-but-curious: they follow the protocol and may try to infer private information from what they observe. Clients may likewise be honest-but-curious and may collude.

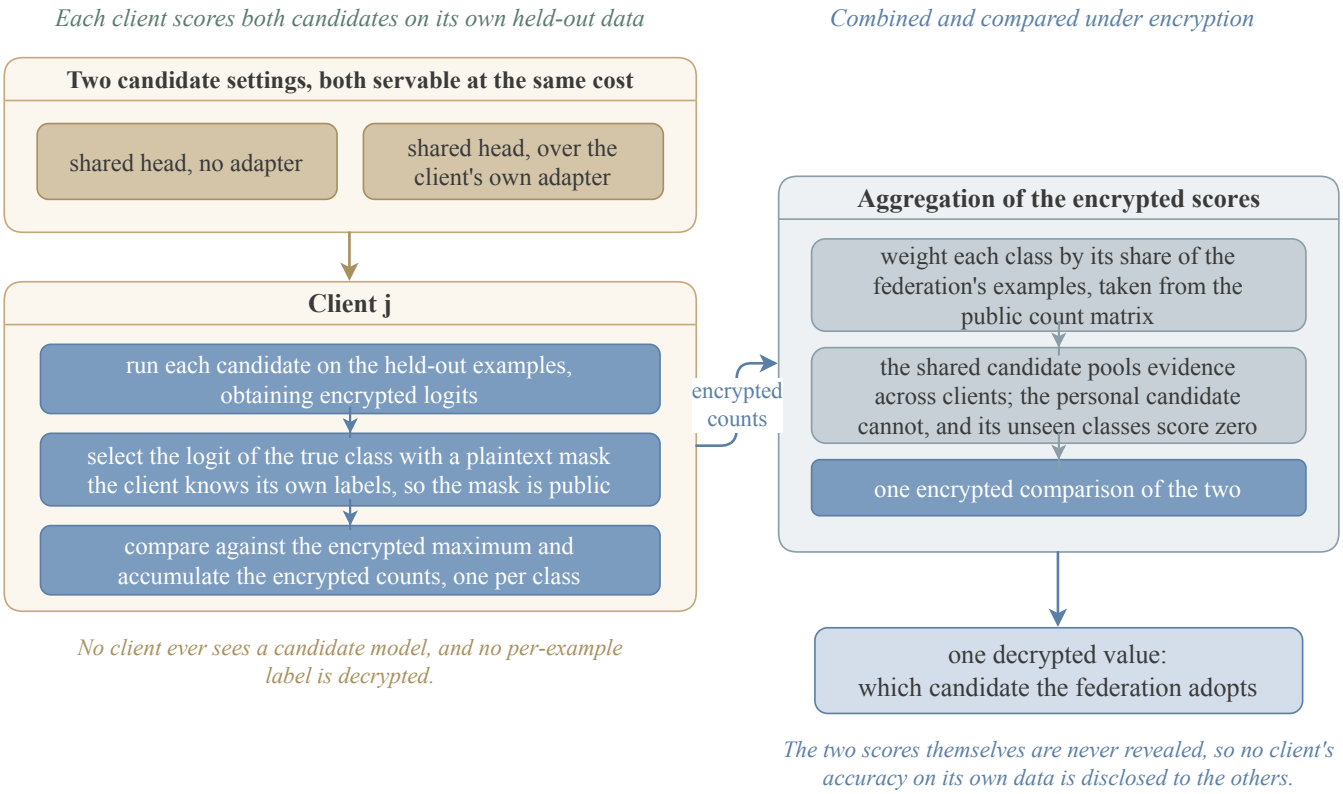

Fig. 3. Choosing between two arrangements without decrypting either. Each client scores both on data held back from its own training, selecting the logit of the true class with a plaintext mask, which is public because the client knows its own labels. The per-class counts are weighted by each class's share of the federation's examples, taken from the count matrix already used for the aggregation. The shared arrangement pools evidence across clients because it is one model; the personal arrangement cannot, and classes a client does not hold contribute nothing, which makes its score a lower bound. Exactly one value is decrypted.

a) View of the server and the serving party.: Ciphertexts under the collective public key, the public per-client sample counts, and the evaluation keys. Neither holds a decryption key, and by section III-C neither can decrypt. The serving party additionally sees encrypted query features, which it cannot read and therefore cannot invert.

b) Collusion among clients.: Decryption is N-out-of-N, so any coalition short of all clients, including a coalition of $N-1$ clients acting together with the server, cannot decrypt. This is the strongest collusion resistance the threshold structure admits, and the design is arranged so that no arithmetic shortcut evades it. No aggregate of the adapters exists, so there is no sum from which a coalition can subtract its own contributions. The shared head is never decrypted, so it cannot be inverted the same way. The per-class totals are never decrypted, so a coalition cannot recover the remaining client's class histogram.

c) Extraction through the query interface.: A client can query the served model and receive labels, and enough such queries reconstruct a functional copy of the shared head by boundary search. This is a bound on queries, not a cryptographic guarantee, and we state it as such. The relevant coalition is again of size $N-1$, since a larger one can decrypt outright, so a per-client query allowance Q constrains the coalition to $(N-1)Q$ queries and must be set below the extraction cost of the deployment's label space. A deployment that cannot enforce such an allowance can add calibrated noise to the returned labels, which composes with the protocol without modification and which we discuss in section V-H; we do not evaluate it, and the accuracy figures we report assume it is not used.

d) Outside the guarantee.: Robustness to actively malicious clients is outside the cryptographic guarantee. A client that uploads a crafted displacement can bias the shared head, and because contributions are encrypted the server cannot inspect them. Section V-H discusses this.

e) Protection without perturbation.: The aggregation of eq. (1) runs entirely under multiparty CKKS, so the displacements are never available in plaintext to the server or to any coalition short of all clients. Nothing is noised. The cryptographic layer therefore costs no accuracy, which is the point on which this design departs from differentially private one-shot federated learning. Section IV states the guarantee, gives an ideal functionality, and separates what the cryptography covers from what the query interface reveals by construction.

IV. Security

This section states what the protocol guarantees. Section IV-A gives an ideal functionality, section IV-B proves that the protocol realizes it against semi-honest parties, and section IV-C bounds what a malicious coalition learns about an honest client's data. Section IV-D shows why the strict

functionality is out of reach against a deviating serving party, and describes two extensions that recover it. Section IV-E separates the guarantees the cryptography provides from the leakage the task itself carries.

A. The Ideal Functionality

We describe what a trusted party would do, so that the protocol can be measured against it. The functionality $\mathcal{F}$ interacts with N clients, an aggregation server, and a serving party.

- **Training.** Each client j sends its dataset $\mathcal{D}_j$. The functionality runs the local training of algorithm 1 and forms the shared head $\theta^\star$ of eq. (1). It stores $\theta^\star$ and sends it to no party.
- **Selection.** The functionality computes the estimator of eq. (2) for both arrangements and announces the index of the larger. It announces nothing else.
- **Serving.** On input (query, x) from client j, if that client has spent fewer than Q queries, the functionality increments its counter and returns $\hat{y} = \arg\max_c(\theta^\star \varphi_j(x))_c$ to client j alone. Otherwise it returns $\perp$.

The functionality also reveals a fixed leakage $\mathcal{L}$ to the adversary: the public parameters, the per-client sample counts n_j, and the index the selection step announces. We write the counts into the leakage because the protocol uses them as public scalars, and we prove security relative to that.

Three properties of $\mathcal{F}$ deserve statement, because they are what the protocol must reproduce. No party ever holds $\theta^\star$. The only value the whole federation learns is one index. A client learns the labels it asks for, and nothing else, up to its allowance.

B. Security Against Semi-Honest Parties

Definition 1 (Semi-honest security)**.** Let $\mathcal{A}$ corrupt a set of parties that may contain the server, the serving party, and at most $N-1$ clients. Write $\mathsf{view}^{\Pi}_{\mathcal{A}}(\{\mathcal{D}_j\}_{j=1}^N)$ for the corrupted parties' joint view of a protocol run on datasets $\{\mathcal{D}_j\}_{j=1}^N$. The protocol Π securely realizes $\mathcal{F}$ with leakage $\mathcal{L}$ if there is a probabilistic polynomial-time simulator $\mathcal{S}$ such that, for every $\{\mathcal{D}_j\}_{j=1}^N$,

$$\mathsf{view}^{\Pi}_{\mathcal{A}}(\{\mathcal{D}_j\}_{j=1}^N) \overset{c}{\approx} \mathcal{S}\big(\mathcal{L},\ \{\mathcal{D}_j\}_{j\ \mathrm{corrupt}},\ \mathsf{out}_{\mathcal{A}}\big),$$

where $\mathsf{out}_{\mathcal{A}}$ collects the answers $\mathcal{F}$ returns to the corrupted clients.

The simulator receives the answers because the protocol delivers them by construction. A definition that withheld them would be unachievable, and the resulting theorem would say nothing about a system that answers questions.

Theorem 1 (Semi-honest security)**.** *Assume the multiparty CKKS scheme is IND-CPA secure and its decryption is N-out-of-N. Then the protocol securely realizes $\mathcal{F}$ with leakage $\mathcal{L}$ against any semi-honest adversary corrupting the server, the serving party, and at most $N-1$ clients, in the sense of definition 1.*

Proof sketch. The simulator is given $\mathcal{L}$, the corrupted clients' datasets, and the answers the corrupted clients receive. It proceeds in three parts.

Setup. The simulator runs the distributed key generation with the corrupted parties, playing the honest clients on uniformly chosen shares. The transcript of that protocol is simulatable by the security of the key-generation protocol, and the resulting collective public key is distributed identically.

Training. For each honest client the simulator sends an encryption of zero in place of $\mathsf{Enc}(g_j \odot \Delta_j)$ and $\mathsf{Enc}(g_j)$. A hybrid argument replaces the honest ciphertexts one at a time. Each neighbouring pair of hybrids is indistinguishable by IND-CPA, which must hold against an adversary that already holds $N-1$ of the N secret-key shares. This is exactly the threshold property of the scheme, and we assume it rather than derive it.

The server's additions and its multiplication by the reciprocal are deterministic functions of those ciphertexts and of public scalars, so the simulator computes them as the server does. The encrypted reciprocal is not such a function. Its inversion circuit consumes levels and restores them by collective refresh, and a collective refresh is an interactive protocol in which every client contributes fresh randomness. The simulator therefore invokes the simulator of the refresh protocol for each of the two refreshes, playing the honest clients on that protocol's simulated shares. Both refreshes occur at fixed points in the circuit that do not depend on any plaintext, so the number and position of these invocations are public and the simulator knows them in advance.

Selection and serving. The selection step decrypts one value. The simulator takes that index from $\mathcal{L}$ and produces the collective key-switching transcript for it by the simulator of the key-switching protocol. For each query of a corrupted client, the simulator holds the answer in $\mathsf{out}_{\mathcal{A}}$ and simulates the key switch to that client in the same way. Queries by honest clients are switched to keys the adversary does not hold, so their transcripts are simulated as key switches of encryptions of zero.

Composing the three parts gives a view computationally indistinguishable from the real one. □

C. Input Privacy Against a Malicious Coalition

Theorem 1 assumes every party follows the protocol. We now ask what a coalition that deviates can learn about the data of a client that does not. We bound what the adversary learns about honest inputs. We do not claim correctness: a client that uploads a crafted displacement can bias the shared head, and section V-H discusses that separately.

Definition 2 (Content game)**.** The adversary $\mathcal{A}$ corrupts the server, the serving party, and $N-1$ clients. One client h stays honest.

1) $\mathcal{A}$ outputs two datasets D_0 and D_1 with $|D_0| = |D_1|$.
2) The challenger samples $b \in \{0,1\}$ and gives D_b to client h.
3) The protocol runs. $\mathcal{A}$ may deviate arbitrarily, and may query the served model up to the allowance of every client it controls.

4) $\mathcal{A}$ outputs b'. Its advantage is $|\Pr[b'=b]-\frac{1}{2}|$.

The equal-size restriction is necessary rather than cosmetic. The per-client sample counts are public, so an adversary free to choose datasets of different sizes reads the answer off the counts without attacking anything.

Theorem 2 (Input privacy)**.** *Assume the conditions of theorem 1, and assume further that every party follows the key-switching protocol, deviating only in the values it uploads and the queries it asks. Then the advantage of $\mathcal{A}$ in definition 2 is at most $\mathrm{negl}(\lambda)+\delta(Q_{\mathrm{tot}})$, where $Q_{\mathrm{tot}}=(N-1)Q$ and δ is the advantage available from Q_{tot} label queries to the served model.*

Proof sketch. The adversary's view consists of the key-generation transcript, the honest client's uploaded ciphertexts, the values the server computes, the key-switching transcripts, and the answers to its own queries. The first four are independent of b up to $\mathrm{negl}(\lambda)$, by the hybrid argument of theorem 1: the honest ciphertexts are indistinguishable from encryptions of zero, and every other value is a function of them and of public scalars. Deviating in the uploaded values does not help, because the adversary already chooses those values in the real protocol and they carry no dependence on b.

What remains is the answers. These do depend on b, because $\theta^\star$ depends on client h's displacement. The dependence is exactly the dependence of a prediction interface on its training data, and it is bounded by what Q_{tot} label queries reveal, which is $\delta(Q_{\mathrm{tot}})$ by definition. □

a) Why the assumption cannot be discharged.: Decryption is a protocol the clients run on a ciphertext the serving party presents. A coalition that presents a ciphertext of its own choosing, rather than the output of algorithm 2, can ask an honest client to help decrypt any value it likes, including a row of $\theta^\star$. The natural repair is to have the honest client refuse such a request. It cannot, and the obstacle is the encryption itself.

Proposition 1 (No plaintext gate from a ciphertext)**.** *Let an honest client decide, by a polynomial-time predicate D on its view, whether to contribute its key-switching share for a presented ciphertext. Let the rest of that view be independent of the underlying plaintext. If the scheme is IND-CPA against an adversary holding $N-1$ key shares, then for every pair of plaintexts m_0 and m_1,*

$$\left|\Pr[D(\mathsf{Enc}(m_0))=1]-\Pr[D(\mathsf{Enc}(m_1))=1]\right|\le\mathrm{negl}(\lambda).$$

Proof. A gap between the two probabilities gives an IND-CPA distinguisher that holds one key share. One share is at most $N-1$ shares, so the assumed security of the scheme applies directly. □

An honest client therefore releases its share for both plaintexts or for neither. It cannot release one for a label and refuse one for a row of $\theta^\star$. No protocol of this message pattern realizes $\mathcal{F}$ against a malicious serving party, so the assumption of theorem 2 is necessary rather than convenient. Without a further mechanism a single deviating request recovers a full ciphertext of plaintext slots, which at ring degree 2^{14} is 8192 real numbers.

Proposition 1 constrains predicates on the *plaintext*. A predicate on the ciphertext escapes it, and section IV-D uses that.

D. Extension to a Malicious Serving Party

Proposition 1 rules out the strict functionality. It does not rule out a weaker one, and it does not rule out checking a ciphertext by its provenance. This subsection describes what the protocol would guarantee under each option and what each one costs. We present them as extensions. The protocol we implement is the one of section III, and no figure in section V includes them.

a) A functionality that is within reach.: Replace the serving step of $\mathcal{F}$ by one that answers with slots rather than with labels. On a request from the serving party that names an arithmetic circuit E over the stored ciphertexts, together with a recipient P, the functionality charges one unit against the allowance, evaluates E, and returns the value in one designated slot of the output to P alone. An honest query is the case in which E is the circuit of algorithm 2. The label then occupies the designated slot, so the two functionalities agree whenever the serving party follows the protocol. A deviating request is a different E, and it returns one scalar of the coalition's choosing.

b) The mechanism that realizes it.: Before it contributes its share, every client multiplies the presented ciphertext by a public plaintext mask that selects a single slot. The mask is fixed and identical for all clients, so they agree on the masked ciphertext without communicating. Shares computed on different ciphertexts do not combine, so one honest client is enough to force every successful decryption to be of a masked ciphertext. The step costs one plaintext-by-ciphertext multiplication, measured at 4.8 ms in table V, and it adds no traffic, because it applies to a ciphertext the clients already receive. Under this mechanism, and assuming each upload carries a proof of knowledge of its plaintext so that a simulator can extract the corrupted clients' inputs, the protocol realizes the functionality above with abort, against a malicious adversary corrupting the server, the serving party and up to $N-1$ clients.

c) Why the allowance still governs.: Each charged request returns one slot, so Q_{tot} requests return at most Q_{tot} scalars. The shared map holds Cd parameters, and one slot carries a few tens of bits of precision. Recovering the map therefore costs on the order of Cd requests, which is the order section V-F measures for label-only extraction. A deviating coalition buys a constant factor rather than a change of order, and the query allowance remains the control.

d) Closing the gap by recomputation.: Proposition 1 blocks predicates on the plaintext. It does not block a client that checks where a ciphertext came from. The serving circuit is a deterministic function of the encrypted head, the encrypted query, the evaluation keys and the public parameters. Its level restoration is deterministic as well, because section V-E restores levels under collectively generated bootstrapping keys rather than by collective refresh. A client can therefore recompute the prescribed output ciphertext and compare it as a string,

which restores $\mathcal{F}$ itself. The design choice that saves $510\,\mathrm{MiB}$ of traffic on every query is the same one that leaves the circuit verifiable.

e) The cost of verifying, and why sampling is enough.: Verifying every query costs one serving evaluation per verifying client, which section V-E puts under two minutes at a hundred classes. Verifying a random fraction p of queries costs p times as much, and the shape of the attack makes a small p sufficient. A coalition that makes k deviating requests escapes a single honest checker with probability at most $(1-p)^k$. Recovering the shared map needs k on the order of Cd, which is 3072 on AG-News and 59 136 on Banking77. At $p = 0.01$ those campaigns escape with probability 4×10^{-14} and below 10^{-250}. The defence is cheap because the attack is long. One deviating request is easy to hide and returns one scalar, and a campaign worth mounting does not survive thousands of independent checks.

f) What none of this covers.: Correctness. A client that uploads a crafted displacement biases the shared head, and no mechanism above detects it. Certifying which function the serving party evaluated, rather than bounding the width of its answer, needs a zero-knowledge proof of correct evaluation or verifiable homomorphic encryption [94]. We implement neither, and section V-H records both limitations.

E. What the Cryptography Does Not Cover

Theorem 1 shows the protocol reveals no more than $\mathcal{F}$ reveals. That leaves the question of what $\mathcal{F}$ itself reveals, and the answer is not nothing.

$\mathcal{F}$ answers queries. Answers carry information about $\theta^\star$, and enough of them reconstruct it. Section V-F measures the rate at three to five queries per parameter of the shared map for a functional copy. This is a property of the functionality, not of our instantiation of it. Any protocol that realizes $\mathcal{F}$ inherits it, and no cryptographic assumption removes it, because the leakage is the service the functionality provides.

Stating it this way separates two claims that are easy to confuse. The cryptographic claim is that the protocol adds nothing to the leakage of $\mathcal{F}$. The operational claim is that $\mathcal{F}$'s own leakage is bounded by the query allowance Q. The first rests on IND-CPA. The second rests on a measurement and on a deployment's ability to meter queries, and section V-F reports both the measurement and the resulting bound on Q.

V. Experiments

Each subsection below states a claim in one sentence, describes the experiment that tests it, and reports the result. The claims are, in order: that a federated head over private representations is a usable model; that its accuracy is competitive with one-shot federated learning on that literature's own partition; that the federation can choose between arrangements without decrypting either; that the cryptographic layer is affordable, including the traffic that recurs; and that what a participant can learn is bounded. Section V-G then states what disclosing the model would have bought, and section V-H records what the protocol does not cover.

TABLE II
Accuracy on the global test set, three seeds, $N = 10$, $\alpha = 0.1$. Both servable arrangements are shown, together with what a client obtains alone and what a disclosed model would reach. The disclosed model is not servable under the threat model of section III-G and is included only as a reference.

		servable			
Task	C	shared head	personal adapter	alone	disclosed
AG-News	4	**0.649**	0.479	0.475	0.739
TREC	6	**0.607**	0.429	0.400	0.711
DBpedia	14	**0.789**	0.754	0.451	0.925
Banking77	77	0.206	**0.686**	0.249	0.760
CIFAR-100	100	0.748	**0.774**	0.197	0.784

A. Setup

Every client fine-tunes on the same frozen public backbone. The trainable unit is a rank-8 low-rank adapter with its down-projection frozen at a shared public initialisation, together with a classifier head. The adapter is trained locally and never transmitted; the head displacement is encrypted and uploaded once. No labelled public data is used anywhere in this paper, because the setting without one is the setting in which an encrypted one-shot protocol is required.

We evaluate on four text classification tasks of increasing label-space size, AG-News with 4 classes, TREC with 6, DBpedia with 14 and Banking77 with 77, using a frozen RoBERTa-base encoder, and on CIFAR-100 with a frozen ViT-B/16. Data are partitioned across clients by a Dirichlet distribution over labels with concentration α, the standard label-skew partition [95], [96], which induces label-distribution skew in the usual taxonomy [23] and holds the feature distribution fixed across clients. Unless stated otherwise there are $N = 10$ clients, $\alpha = 0.1$, the local trajectory is 200 steps, and every number is the mean over three seeds. Each client reserves at least one held-out example from every class it holds, as section III-F requires, and tops the held-out set up to one tenth of its shard at random.

Two reference points appear throughout. The *local* model is what a client obtains alone, with its own adapter and its own head and no federation at all. The *disclosed* model is the one this protocol declines to build: both the adapter and the head aggregated and decrypted, so that every party holds it in plaintext. The disclosed model is not a configuration of our protocol and is not servable under the threat model of section III-G; it is reported so that the price of never disclosing a model can be read off directly.

B. Utility of the Federated Head

a) Claim.: Sharing only the classifier head, over representations that each client adapts privately, yields a model usable across the whole label space, and the preferable arrangement depends on the size of the label space.

Table II reports both servable arrangements and both reference points. The arrangement labelled *shared head* gives every client the same model over the bare public backbone. The arrangement labelled *personal adapter* gives each client the shared head over its own privately adapted representation.

Three observations follow. First, the federation accounts for most of the resulting accuracy. A client alone reaches 0.20 to 0.48, because under this skew it holds only a few classes and cannot classify the rest, while the federated head reaches 0.61 to 0.79 on four of the five tasks. The gap is largest exactly where a client's own data covers least of the label space.

Second, the preferable arrangement changes with the size of the label space, and the mechanism is identifiable. On the small label spaces the shared head wins by a wide margin, 0.17 on AG-News and 0.18 on TREC. On the large ones the personal adapter wins, and on Banking77 it wins decisively: the shared head collapses to 0.206 while the personal adapter holds 0.686. The collapse is a coverage failure. With 77 classes and this skew most head rows are decided by very few clients or none, and a head over the bare public backbone has nothing else to work with. A privately adapted representation improves separability for every class the client will ever be asked about, including those it never observed, which is what rescues the large label spaces.

Third, the two failure modes are complementary rather than ordered by severity. The shared head fails when coverage is thin. The personal adapter fails when each client's representation drifts far enough that a common head no longer transfers, which is what occurs when the local task approaches a single class. Neither arrangement dominates the other, so the federation must select between them, which is the subject of section V-D.

b) Sensitivity.: Table III varies the protocol axes around the default on DBpedia. Accuracy falls with skew as expected, from 0.971 at $\alpha = 1.0$ to 0.789 at $\alpha = 0.1$, and the 0.812 at $\alpha = 0.05$ is within the seed spread at that setting, which runs from 0.769 to 0.861. Longer local training helps, from 0.718 at 100 steps to 0.870 at 400, and it helps the shared head more than the personal adapter, since the head over the bare backbone gains 0.27 across that range against 0.06 for the personal adapter.

The more informative reading concerns which arrangement is selected. The estimator chooses the shared head at $\alpha \in \{0.05, 0.1\}$ and the personal adapter at $\alpha \in \{0.3, 1.0\}$, on every seed in each case. The crossover is therefore not a property of the label space alone, as section V-B might suggest, but of coverage: the personal adapter is preferable once each client sees enough of the label space for its own representation to transfer, and the shared head is preferable when it does not. The estimator locates that crossover from encrypted local evidence, with no test set and no fitted threshold.

C. Comparison with One-Shot Federated Learning

a) Claim.: The protocol gives up no accuracy for protecting the contributions, where the differentially private one-shot methods give up an amount that grows as their budget tightens, and it withholds the model at a cost those methods do not incur because they do not offer the property.

The peer group evaluates on different tasks from ours, so we ran our protocol on theirs rather than restate numbers from incomparable setups. We adopt the configuration DENSE reports [56], CIFAR-10 across $N = 5$ clients under a Dirichlet partition, and report the arrangement our estimator selects.

TABLE III
SENSITIVITY ON DBPEDIA ALONG THE PROTOCOL AXES, EACH VARIED AROUND THE DEFAULT OF $N = 10$, $\alpha = 0.1$ AND 200 LOCAL STEPS. THE ROW REPORTS THE ACCURACY OF THE ARRANGEMENT THE ESTIMATOR SELECTS. THE SKEW ROW IS THE MEAN OVER THREE SEEDS; THE LOCAL-STEPS ROW IS A SINGLE SEED, AND ITS DEFAULT CELL IS THAT SEED'S VALUE RATHER THAN THE THREE-SEED MEAN. THE CLIENT-COUNT ROW IS IN PROGRESS.

Label skew α				
	0.05	0.10	0.30	1.00
selected arrangement	0.812	0.789	0.914	0.971
selected	shared	shared	personal	personal
Clients N				
	10	20	50	
selected arrangement	0.789	??	??	
Local steps				
	100	200	400	
selected arrangement	0.718	0.803	0.870	

b) Why absolute accuracy does not settle the comparison.: On that partition our selected arrangement reaches 0.948 at $\alpha = 0.1$, against the 0.503 that DENSE reports and the 0.406 of FedDF. We do not present that margin as a result. The methods above train a ResNet-18 from scratch, whereas every client here fine-tunes a frozen public ViT-B/16, and the representation accounts for essentially all of the difference. Any of these methods would gain comparably from the same backbone, and running them on it is not a comparison either paper's authors chose. Absolute accuracy across papers therefore measures the backbone.

c) The comparison that does hold.: What compares across papers is the accuracy each method gives up for its privacy mechanism, since every paper measures that against its own unprotected baseline on its own architecture, and the backbone cancels. Read that way the peer group separates into three cases.

DENSE and FedDF give up nothing, because they protect nothing. Their single contribution is exposed in plaintext, and by the evidence of section II-A that contribution is the quantity worth attacking.

FedAUXfdp does protect its contribution, and its cost is governed by the budget. Against its own undefended baseline of 0.752 on CIFAR-10 at $N = 20$ and $\alpha = 0.16$, the published figures give up 0.006 at $\varepsilon = 1.0$ and 0.029 at $\varepsilon = 0.5$, which is affordable, then 0.413 at $\varepsilon = 0.1$ and 0.626 at $\varepsilon = 0.01$ [61]. The mechanism is inexpensive only while the budget is loose, and the accuracy is spent on the artifact that is released.

Our contribution is protected cryptographically, and the cost is zero by construction rather than by measurement, since the decrypted result matches the plaintext computation to the encoding precision of the scheme. Withholding the model is a separate charge, and it is one no method above incurs, because every one of them ends by handing a plaintext model to the participants. It costs 0.007 on this task and 0.03 to 0.14 across table II. The comparison to draw is therefore between a mechanism whose price rises as its guarantee tightens and one whose price is fixed and paid for a different property.

One figure bounds the federation's own contribution independently of the backbone, because it is measured on the same

TABLE IV
CHOOSING BETWEEN THE TWO SERVABLE ARRANGEMENTS WITHOUT DECRYPTING EITHER. CELLS ARE COUNTED OVER FIVE TASKS AND THREE SEEDS. REGRET IS THE ACCURACY GIVEN UP BY A WRONG CHOICE, AVERAGED OVER TASKS.

Selection rule	correct cells	regret
majority of held-out accuracies	6/15	0.383
same, class-balanced within a client	6/15	0.383
global-prior estimator, zero fill	12/15	0.027
global-prior estimator, rare-class fill	**13/15**	**0.019**

TABLE V
PER-OPERATION COST. THE FIRST TWO ROWS ARE THE SAME OPERATION WITH THE QUERY ENCRYPTED AND LEFT IN PLAINTEXT, WHICH IS WHAT ENCRYPTING THE QUERY COSTS.

Operation	cost
head applied to an encrypted query	35.2 ms
head applied to a plaintext query	4.8 ms
ciphertext addition	0.7–1.1 ms
rotation	30.8 ms
selection mask and accumulate	1.1 ms
key switch to the querier, $N = 10$	181 ms
reciprocal under encryption, once per aggregation	4.21 s

backbone at both ends. A client working alone on this partition reaches 0.364, against 0.948 for the selected arrangement. That difference is what the protocol supplies.

D. Accuracy of the Selection Rule

a) Claim.: The direct procedure for choosing between the two arrangements fails systematically, and the estimator of section III-F corrects it while disclosing only the index of the selected arrangement.

We first establish that the direct procedure fails, since this is what motivates the estimator. Each client measures both arrangements on data held back from its own training and the federation takes the sample-weighted majority. Across all fifteen cells of table II, this procedure selects the personal adapter every time, including on AG-News and TREC where the personal adapter is 0.17 and 0.18 worse. Weighting each class equally within a client's held-out set does not help, and selects identically.

The failure is structural rather than statistical. The shared arrangement is one model, and the union of the clients' data is the whole training set, so pooled held-out measurements estimate its accuracy on the global distribution to within 0.02. The personal arrangement is N different models, and client j's held-out data measures only client j's own model, on client j's own distribution, which is the distribution that model was fitted to. Its measured accuracy exceeds its true global accuracy by 0.29 on average. A client's held-out set also contains only the classes that client holds, so the very quantity that decides the comparison, accuracy on classes the client has never seen, cannot be measured locally at all. The procedure compares an accurate number against an optimistic one, so it is not close to correct and it does not become correct with more held-out data.

Table IV reports the estimator of eq. (2) against that baseline. Scoring both arrangements under the federation's own class prior, letting the shared arrangement pool evidence across clients and scoring the personal arrangement's unobserved classes at zero, selects correctly in 12 of 15 cells. Filling the unobserved classes from each client's rarest classes instead of with zero selects correctly in 13. Both are parameter-free in the sense that matters: the rule compares two numbers on the same absolute accuracy scale, with no fitted threshold.

Two properties of the estimator merit separate mention. The decisive case is Banking77, where choosing wrongly costs 0.48; the estimator selects correctly on all three seeds, by margins of 0.13 to 0.24. And the estimator responds to the data rather than to the label space: on the AG-News seed where an unlucky partition leaves the shared head at 0.402 and the personal adapter genuinely better, it switches to the personal adapter, which a rule keyed to the number of classes would not do.

The estimator yields a calibrated accuracy estimate, not only a ranking. Its estimate of the shared arrangement's global accuracy is within 0.019 on average across all fifteen cells, computed entirely from encrypted local evidence with no access to a test set.

b) Disclosure of the estimator.: One value is decrypted for the whole federation, the index of the arrangement adopted. The per-client per-class accuracies are not decrypted, which matters: together with the count matrix they would describe each client's label distribution alongside its model quality.

E. Cryptographic Cost

a) Claim.: The cryptographic cost of the protocol is dominated by one-time setup, and the recurring per-query traffic is a few megabytes.

We do not simulate the cryptography. The protocol is implemented in real multiparty CKKS on Lattigo [97], [88]: distributed key generation, encryption under the collective public key, the head aggregation, the reciprocal under encryption, the encrypted argmax, and key switching to a designated party. All figures use ring degree 2^{14} for the shallow operations and a deeper chain at ring degree 2^{15} for the circuits that need it. Timings are single-run wall clock on a commodity CPU and are reported as indicative; communication figures are exact.

b) Cost of query encryption.: Applying the head to an encrypted query costs 35.2 ms against 4.8 ms for a plaintext one, a factor of seven. That factor buys the property that the serving party never sees the feature vector and therefore cannot invert it to recover the client's input, and it is small beside the argmax that follows.

c) The encrypted reciprocal.: Inverting the per-class totals under encryption, which section III-D requires so that no coalition can read the remaining client's class histogram, takes 4.21 s at $N = 10$ and needs two collective refreshes. It agrees with the plaintext reciprocal to a relative error of 1.9×10^{-8}, so it costs no accuracy. The figure grows slowly with the federation, 3.16 s at $N = 5$ and 6.30 s at $N = 20$, because only the refreshes involve the clients. This is the one deep operation in building the shared head, and it is paid once

TABLE VI
COST OF THE N-OUT-OF-N STEPS AGAINST THE SIZE OF THE FEDERATION, AT RING DEGREE 2^{14}. THE KEY SWITCH IS THE OPERATION THAT RETURNS A LABEL TO THE QUERIER; IT IS PAID PER QUERY. THE RECIPROCAL IS PAID ONCE PER AGGREGATION.

	$N = 5$	$N = 10$	$N = 20$
key switch to the querier	95 ms	181 ms	361 ms
key-switching shares	10.0 MiB	20.0 MiB	40.0 MiB
encrypted reciprocal	3.16 s	4.21 s	6.30 s

per aggregation rather than per query, so a few seconds is immaterial beside the local training it follows.

d) The argmax.: The argmax is the one operation in the protocol that is not shallow. Evaluated as a sequential fold of $C - 1$ comparisons it takes about 16 s per comparison, which is roughly twenty minutes at a hundred classes. Packing the logits into one ciphertext and reducing them by a tournament lowers the number of sequential comparisons from $C - 1$ to $\lceil \log_2 C \rceil$ and brings the same computation to under two minutes at a hundred classes, a fourteenfold reduction, and the result is exact rather than approximate. Because homomorphic evaluation under a collectively generated key is identical to evaluation under a single key, single-key results carry over, and GPU implementations of the same reduction report a full-vocabulary argmax in well under a second [80].

e) Threshold decryption against the size of the federation.: Producing a label requires a share from every client, so the cost of the final step grows with the federation. Table VI reports it. Both the time and the traffic are linear in N, at 18 ms and 2.0 MiB per participating client, because each contributes one independent share and the serving party only sums them. The encrypted reciprocal grows more slowly, since only its two refreshes involve the clients.

f) Hardware acceleration.: The figures above are single-threaded CPU, and the argmax is the only operation slow enough to matter, so the question is what specialised hardware does to it. Since the cost of a CKKS operation grows with the ring degree, the comparison is only meaningful at a fixed one. Our implementation costs 37.3, 78.1 and 166.7 ms for a ciphertext-by-ciphertext product at ring degrees 2^{14}, 2^{15} and 2^{16}, and 31.7, 67.1 and 144.1 ms for a rotation. Microbenchmarks of the same primitives under a CUDA implementation of CKKS, at ring degree 2^{16}, put the product at 30 ms and the rotation at 29.5 ms, so the ratio at matched ring degree is about 5 for both. Two things make that a conservative figure. The GPU numbers are taken at a deeper level budget than ours, and a deeper budget makes every operation more expensive. They are also measured on a mid-range inference card, whereas the GPU homomorphic-encryption libraries are tuned for current datacentre parts, on which published figures are substantially better.

The projection from a per-primitive ratio to the protocol is unusually safe here, and the reason is the shape of the circuit rather than the quality of the measurement. Serving a query is a fixed sequence: one ciphertext-by-ciphertext product, then $\lceil \log_2 C \rceil$ rounds of comparison, then one key switch. There is no data-dependent control flow, no attention pattern whose cost depends on the input, and no scheduling problem across layers, so a speedup on the primitives carries through to the whole rather than being absorbed by effects that dominate in deep encrypted inference. We none the less stop at the ratio rather than quoting an end-to-end accelerated latency, because the argmax is bootstrap-bound and reported bootstrapping times on GPUs span more than two orders of magnitude with the ring degree, the packing and the implementation. A single number drawn from that range would say more about which number was drawn than about this protocol.

TABLE VII
COMMUNICATION AT $N = 10$, RING DEGREE 2^{14}. SHARE SIZES DO NOT DEPEND ON THE NUMBER OF CLIENTS, SO ONLY THE TOTALS SCALE. THE RECURRING ROW IS THE ONE THAT MATTERS FOR A DEPLOYMENT.

When	Item	Per client
setup, once	collective public key share	1.13 MiB
	relinearization key share	27.0 MiB
	rotation keys, seven of them	63.0 MiB
training, once	encrypted head displacement	a few MiB
per query	encrypted features, uploaded	2.0 MiB
	encrypted label, returned	0.5 MiB
	key-switching shares, ten	2.5 MiB
	total per query	**5.0 MiB**

g) Communication.: The protocol is one-shot in the sense of section I: no intermediate training artifact is ever exposed. It is not silent. Key generation precedes it, selection costs a bounded exchange, and serving costs traffic per query, so we account for all of it in table VII.

One design decision governs the recurring figure, and we state it explicitly because the alternative is two orders of magnitude more expensive. The argmax spends its levels and must have them restored several times per query. Restoring them by a collective refresh would require a share from every client for every refresh, which at a hundred classes amounts to roughly 510 MiB of traffic for a single label. Generating bootstrapping keys once, collectively, and letting the serving party restore levels on its own reduces that traffic to zero, because evaluation under a collectively generated key is identical to evaluation under a single key. The protocol therefore specifies the latter, at a one-time cost of 15.5 MiB of key material at ring degree 2^{16}, generated collectively in 49 s. One key generation of 15.5 MiB therefore replaces 510 MiB on every subsequent query.

h) Correctness.: The decrypted result matches the plaintext computation to a relative error at the level of the encoding precision of the scheme, and the encrypted argmax is exact: over label spaces from four to a hundred classes it returns the same index as the plaintext argmax in every case tested. The accuracy figures of table II are therefore unaffected by the cryptography, which is the sense in which encryption costs no accuracy here.

F. Residual Leakage

a) Claim.: The protocol exposes no training-time artifact and no model, so the only remaining channel is the sequence of

TABLE VIII
LABEL-ONLY EXTRACTION OF THE SERVED HEAD: MEAN FIDELITY OVER THREE SEEDS AND BOTH ARRANGEMENTS, AGAINST THE NUMBER OF QUERIES SPENT. THE MAJORITY COLUMN IS THE SHARE OF THE MOST COMMON LABEL, WHICH IS WHAT A COPY ACHIEVES FOR FREE.

Task	C	majority	10^3	5×10^3	2×10^4	5×10^4	2×10^5
AG-News	4	0.488	0.635	0.823	0.936	0.973	0.993
DBpedia	14	0.250	0.387	0.612	0.804	0.901	0.971
Banking77	77	0.083	0.174	0.355	0.592	0.755	0.900

answers the served model returns, and that channel is bounded by the query allowance rather than eliminated.

The training-time surface is removed by construction. No gradient, update, or data-derived summary is ever available in plaintext, there is one exchange rather than many, and its single artifact is a ciphertext. The model is removed as well: no party holds it in plaintext at any point, so the white-box attacks that a released model admits have no object to attack. What remains is a client that can ask the served model questions and read labels.

Two distinct bounds apply to this channel, and they should not be conflated. The first is cryptographic and is stated in theorem 1: the protocol's messages are computationally independent of the private data. The second is not cryptographic. A client that queries the served model at points of its own choosing can, with enough queries, reconstruct a functional copy of the shared head. Returning only the label rather than the logits denies the linear solve that would otherwise recover the head in as many queries as the feature dimension, which raises the cost but does not make it infinite. We therefore state the guarantee as a bound on queries, and we measure what that bound has to be.

b) Cost of extraction.: We give the adversary every advantage the protocol admits. It is a participating client, so it computes query features itself and may submit an arbitrary vector rather than a real input, and it is charged one query per label. It fits a linear model to the labelled queries and we score the copy by *fidelity*, the rate at which it agrees with the served model on held-out features. Table VIII reports the result.

Extraction is possible and its cost is orderly. Reaching fidelity 0.90 takes about 1.2×10^4 queries on AG-News, 5.0×10^4 on DBpedia and 2.0×10^5 on Banking77. Those figures track the size of the head rather than the task: the head has Cd parameters, and in each case fidelity 0.90 costs between three and five times that number of queries, with 0.80 costing roughly 1.5 times and 0.95 roughly ten times. A deployment can therefore set its allowance from C and d without repeating this experiment.

The allowance follows. A coalition of $N-1$ clients, the largest that cannot decrypt, commands $(N-1)Q$ queries, so holding it below fidelity 0.90 at $N = 10$ requires Q below roughly 1.4×10^3 per client on AG-News and 2.2×10^4 on Banking77. The binding case is the small label space, not the large one, which is worth stating plainly because it runs against intuition: a head with few rows is cheap to copy. It is also the case in which the shared arrangement is preferable (table II), so the deployments that gain most from the federation are the ones that must meter queries most tightly.

One methodological point. We also ran a boundary-search variant that spends half its budget bisecting toward the decision boundary, on the expectation that points near a boundary are more informative. It is consistently *worse*, reaching 0.592 against 0.755 on Banking77 at 5×10^4 queries, because a bisection chain returns many nearly identical points and a linear fit gains less from them than from the same number of independent ones. We report the stronger of the two, which is also the simpler.

c) Comparison with a released model.: The contrast is not a matter of degree. A participant holding the model in plaintext has fidelity 1 at zero queries, may read the parameters directly rather than infer them, and is subject to no allowance because there is nothing to meter. Granting logits rather than labels would be nearly as generous: the head is linear in features the client computes itself, so $d+1$ logit queries recover it exactly by linear solve, which is 769 queries here against the 1.2×10^4 to 2.0×10^5 that labels demand. We confirm this directly, recovering the head to fidelity 1.000 from 10^3 answers when the served interface returns a distribution instead of an index. Restricting the answer to the label is therefore what converts extraction from a solve into a search, and the query allowance is what prices that search. The same conclusion has been reached from the other direction in deployment, where recovering a production model's final projection layer was made materially harder by withdrawing exactly the richer outputs this protocol never offers [90].

G. Comparison with Model Disclosure

Never disclosing the model costs accuracy, and table II allows the price to be read directly. Against a model that is aggregated, decrypted and handed to every participant, the servable arrangement selected by section V-D gives up 0.07 on AG-News, 0.10 on TREC, 0.14 on DBpedia, 0.08 on Banking77 and 0.03 on CIFAR-100. The price is largest in the middle of the label-space range and smallest at the extremes, and it is paid for a property that no accuracy figure can substitute for: after the protocol ends, there is no plaintext model anywhere.

We do not present the disclosed model as an alternative configuration of this protocol, since it does not satisfy the requirements of section III-A. Disclosing the model changes the threat model rather than the operating point. A participant that holds the model in plaintext can inspect it, run inference on it without limit, redistribute it, and mount the white-box attacks that section II-A surveys, and the query allowance of section V-F becomes meaningless. Those are different guarantees and they answer a different question. A deployment that can disclose the model should use a protocol designed for that, and prior one-shot federated learning provides several. The construction presented here addresses deployments in which disclosure is not permitted.

H. Scope and Limitations

a) Linearity of the shared quantity.: This is what section III-D shows the never-disclosed requirement forces, and

it is a real restriction. A shared adapter inside the backbone would adapt the representation for every participant rather than only for its owner, and table II shows on the small label spaces what a privately adapted representation gives up. Serving such an adapter reduces to secure transformer inference (section II-E) and composes with that literature at its cost; we do not attempt it here.

b) Bounds on model extraction.: The query allowance of section V-F is an operational control. Where honest use is intrinsically high-volume, so that no allowance separates it from extraction, calibrated noise added to the returned labels bounds cumulative leakage instead [98]. That mechanism composes with the protocol without modification, and it is orthogonal to it: the protocol protects the contributions cryptographically, and differential privacy would protect the answers statistically. We do not evaluate it, and every accuracy figure in this paper assumes it is not in use.

c) Malicious participants.: The cryptographic guarantees assume honest-but-curious parties. A client that uploads a crafted head displacement can bias the shared head, and because contributions are encrypted the server cannot inspect them. Two directions are compatible with the encryption and are left to future work: an upload-time norm bound enforced by a zero-knowledge proof, which caps any single client's influence without revealing its contribution, and robust aggregation evaluated homomorphically. The one-shot structure shrinks the attack surface to a single exchange but removes the cross-round consistency checks a multi-round protocol could use.

d) Availability.: Threshold decryption is N-out-of-N, which gives the strongest collusion resistance the structure admits and the weakest availability: every client must participate for a label to be produced. A t-out-of-N threshold would relax this, at the cost of admitting coalitions of size t and requiring a tighter query allowance, since the bound of section V-F scales with the size of the largest coalition that cannot decrypt. Which point on that trade-off a deployment wants is a deployment question.

VI. Conclusion

We presented HE-OFT, a federated fine-tuning protocol in which the trained model is never disclosed. The design follows from the observation that federated learning's privacy risk is concentrated in the quantities a protocol exposes while the model is being built, and from the further requirement that in some deployments the finished model may not be distributed either. Each client fine-tunes on a frozen public backbone, retains its adapter locally, and uploads a single encrypted head displacement. The server combines those displacements under multiparty CKKS and never decrypts the result; queries are answered under encryption, and a quorum of clients returns only the predicted label, addressed to the party that asked.

Withholding the model constrains what may be shared. Since the querying party must form its query from components it can evaluate itself, those components must be public, and the shared trained quantity must therefore attach after the last nonlinearity of the network. This is the source of the design's three parts: the partition of the trainable unit, inference on the encrypted head, and selection between candidate arrangements that no party can observe. Across four text classification tasks and one vision task, the protocol produces a model substantially stronger than any client obtains alone, at a cost of 0.03 to 0.14 accuracy relative to a model that is aggregated and disclosed. The cryptographic layer itself costs no accuracy, and the recurring communication is a few megabytes per query.

Two directions follow. Robustness to actively malicious participants is outside the present guarantee, and reconciling robust aggregation with an encrypted contribution remains open; an upload-time norm bound enforced by a zero-knowledge proof is the most direct route. Serving a shared quantity that sits inside the backbone, rather than after it, reduces to secure transformer inference, and the accuracy that would recover is quantified in table II.

Acknowledgments

We acknowledge the support of TÜBİTAK (the Scientific and Technological Research Council of Türkiye) projects 124E091 and 124N941. The authors used Claude Sonnet 4.6 and Claude Opus 4.6 to enhance the manuscript by shortening sentences, correcting typos, and improving grammar.

References

[1] R. Bommasani, D. A. Hudson, E. Adeli *et al.*, "On the opportunities and risks of foundation models," *arXiv preprint arXiv:2108.07258*, 2021.

[2] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen, "LoRA: Low-rank adaptation of large language models," in *International Conference on Learning Representations (ICLR)*, 2022.

[3] European Parliament and Council of the European Union, "Regulation (eu) 2016/679 (general data protection regulation)," Official Journal of the European Union, L 119, pp. 1–88, 2016, art. 5(1)(b–c) purpose limitation and data minimisation; Art. 9 special categories; Chapter V transfers to third countries.

[4] U.S. Department of Health and Human Services, "HIPAA privacy rule," 45 C.F.R. Parts 160 and 164, 2013, § 164.502(b) minimum necessary; § 164.514(a–b) de-identification (Expert Determination and Safe Harbor); § 164.508 authorization; § 164.502(e) business-associate agreements.

[5] Republic of Türkiye, "Law on the protection of personal data (kişisel verilerin korunması kanunu), law no. 6698," 2016, art. 6 special categories of personal data; Art. 5 processing conditions; Art. 9 transfer abroad (as amended by Law No. 7499, 2024).

[6] Standing Committee of the National People's Congress, "Personal information protection law of the people's republic of china," 2021.

[7] Parliament of India, "Digital personal data protection act," 2023.

[8] G. Xiao, J. Lin, and S. Han, "Offsite-tuning: Transfer learning without full model," *arXiv preprint arXiv:2302.04870*, 2023.

[9] L. Zhu, Z. Liu, and S. Han, "Deep leakage from gradients," in *NeurIPS*, vol. 32, 2019.

[10] J. Geiping, H. Bauermeister, H. Dröge, and M. Moeller, "Inverting gradients – how easy is it to break privacy in federated learning?" in *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.

[11] M. Nasr, R. Shokri, and A. Houmansadr, "Comprehensive privacy analysis of deep learning: Passive and active white-box inference attacks against centralized and federated learning," in *IEEE S&P*, 2019, pp. 739–753.

[12] F. Boenisch, A. Dziedzic, R. Schuster, A. S. Shamsabadi, I. Shumailov, and N. Papernot, "When the curious abandon honesty: Federated learning is not private," in *2023 IEEE 8th European Symposium on Security and Privacy (EuroS&P)*. IEEE, 2023, pp. 175–199.

[13] L. Fowl, J. Geiping, W. Czaja, M. Goldblum, and T. Goldstein, "Robbing the fed: Directly obtaining private data in federated learning with modified models," in *International Conference on Learning Representations (ICLR)*, 2022.

[14] S. Sav, A. Pyrgelis, J. R. Troncoso-Pastoriza, D. Froelicher, J.-P. Bossuat, J. Sá Sousa, and J.-P. Hubaux, "POSEIDON: Privacy-preserving federated neural network learning," in *NDSS*, 2021.

[15] European Parliament and Council of the European Union, "Regulation (eu) 2024/1689 (artificial intelligence act)," Official Journal of the European Union, L, 2024/1689, 2024, art. 10 data and data governance for high-risk AI systems; Annex III; Art. 6(1) with Annex I.

[16] B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. A. y. Arcas, "Communication-efficient learning of deep networks from decentralized data," in *AISTATS*, 2017.

[17] O. Zari, C. Xu, and G. Neglia, "Efficient passive membership inference attack in federated learning," arXiv:2111.00430; poster, NeurIPS Workshop on Privacy in Machine Learning, 2021.

[18] L. Melis, C. Song, E. De Cristofaro, and V. Shmatikov, "Exploiting unintended feature leakage in collaborative learning," in *2019 IEEE Symposium on Security and Privacy (S&P)*. IEEE, 2019.

[19] H. Takahashi, J. Liu, and Y. Liu, "Breaching FedMD: Image recovery via paired-logits inversion attack," in *CVPR*, 2023, pp. 12 198–12 207.

[20] Z. Yang, Y. Zhao, and J. Zhang, "FD-Leaks: Membership inference attacks against federated distillation learning," in *APWeb-WAIM*, ser. Lecture Notes in Computer Science, vol. 13423. Springer, 2023, pp. 364–378.

[21] Y. Gu and Y. Bai, "LDIA: Label distribution inference attack against federated learning in edge computing," *Journal of Information Security and Applications*, vol. 74, p. 103475, 2023. [Online]. Available: https://dl.acm.org/doi/10.1016/j.jisa.2023.103475

[22] H. Shi, T. Ouyang, and A. Wang, "Unveiling client privacy leakage from public dataset usage in federated distillation," *PoPETs*, vol. 2025, no. 4, pp. 201–215, 2025.

[23] P. Kairouz, H. B. McMahan, B. Avent *et al.*, "Advances and open problems in federated learning," *Foundations and Trends in Machine Learning*, vol. 14, no. 1–2, pp. 1–210, 2021.

[24] R. Shokri, M. Stronati, C. Song, and V. Shmatikov, "Membership inference attacks against machine learning models," in *IEEE S&P*, 2017, pp. 3–18. [Online]. Available: https://arxiv.org/abs/1610.05820

[25] S. Yeom, I. Giacomelli, M. Fredrikson, and S. Jha, "Privacy risk in machine learning: Analyzing the connection to overfitting," in *31st IEEE Computer Security Foundations Symposium (CSF)*, 2018, pp. 268–282. [Online]. Available: https://arxiv.org/abs/1709.01604

[26] A. Salem, Y. Zhang, M. Humbert, P. Berrang, M. Fritz, and M. Backes, "ML-Leaks: Model and data independent membership inference attacks and defenses on machine learning models," in *Network and Distributed System Security Symposium (NDSS)*, 2019. [Online]. Available: https://arxiv.org/abs/1806.01246

[27] N. Carlini, S. Chien, M. Nasr, S. Song, A. Terzis, and F. Tramer, "Membership inference attacks from first principles," in *IEEE S&P*, 2022, pp. 1897–1914. [Online]. Available: https://arxiv.org/abs/2112.03570

[28] M. Fredrikson, S. Jha, and T. Ristenpart, "Model inversion attacks that exploit confidence information and basic countermeasures," in *Proceedings of the 22nd ACM SIGSAC Conference on Computer and Communications Security (CCS)*, 2015, pp. 1322–1333. [Online]. Available: https://dl.acm.org/doi/10.1145/2810103.2813677

[29] B. Hitaj, G. Ateniese, and F. Pérez-Cruz, "Deep models under the GAN: Information leakage from collaborative deep learning," in *ACM CCS*, 2017, pp. 603–618.

[30] K. Bonawitz, V. Ivanov, B. Kreuter, A. Marcedone, H. B. McMahan, S. Patel, D. Ramage, A. Segal, and K. Seth, "Practical secure aggregation for privacy-preserving machine learning," in *ACM CCS*, 2017, pp. 1175–1191.

[31] J. So, R. E. Ali, B. Güler, J. Jiao, and A. S. Avestimehr, "Securing secure aggregation: Mitigating multi-round privacy leakage in federated learning," in *AAAI*, vol. 37, no. 8, 2023, pp. 9925–9933.

[32] R. Kerkouche, G. Ács, and M. Fritz, "Client-specific property inference against secure aggregation in federated learning," in *WPES*. ACM, 2023, pp. 15–30.

[33] K. Garimella, N. K. Jha, and B. Reagen, "Sisyphus: A cautionary tale of using low-degree polynomial activations in privacy-preserving deep learning," in *PPML Workshop, CCS*, 2021.

[34] M. Baruch, N. Drucker, L. Greenberg, and G. Moshkowich, "A methodology for training homomorphic encryption friendly neural networks," in *ACNS Workshops*, ser. Lecture Notes in Computer Science, vol. 13285. Springer, 2022, pp. 536–553.

[35] P. Agamennone, "Polynomial approximations of relu for secure cnn inference in homomorphic encryption environments," *IEICE Transactions on Fundamentals of Electronics, Communications and Computer Sciences*, vol. E108.A, no. 7, 2025.

[36] F. Al Hossain and T. Rahman, "A training framework for optimal and stable training of polynomial neural networks," *arXiv preprint arXiv:2505.11589*, 2025.

[37] A. Ibarrondo and M. Önen, "FHE-compatible batch normalization for privacy-preserving deep learning," in *Privacy in Machine Learning Workshop (PriML), NeurIPS*, 2021. [Online]. Available: https://www.eurecom.fr/en/publication/5626

[38] C. Zhang, S. Li, J. Xia, W. Wang, F. Yan, and Y. Liu, "Batchcrypt: Efficient homomorphic encryption for cross-silo federated learning," in *USENIX ATC*. USENIX Association, 2020, pp. 493–506.

[39] W. Jin, Y. Yao, S. Han, J. Gu, C. Joe-Wong, S. Ravi, S. Avestimehr, and C. He, "Fedml-he: An efficient homomorphic-encryption-based privacy-preserving federated learning system," in *NeurIPS*, vol. 36, 2023.

[40] X. Wei, R. Huang, L. Lei, and J. Wu, "Fedshe–cq: A communication-efficient homomorphic encryption framework for federated learning," *Computer Networks*, vol. 260, p. 111813, 2025.

[41] J. Li *et al.*, "SHE-LoRA: Selective homomorphic encryption for federated tuning with heterogeneous LoRA," *arXiv preprint arXiv:2505.21051*, 2025.

[42] Kemmaka and Tran, "Hyb-Agg: Communication-efficient secure aggregation via hybrid multi-key homomorphic encryption and masking," *arXiv:2511.23252*, 2025.

[43] S. Lee, G. Lee, J. W. Kim, J. Shin, and M.-K. Lee, "HETAL: Efficient privacy-preserving transfer learning with homomorphic encryption," in *International Conference on Machine Learning (ICML)*, 2023, arXiv:2403.14111.

[44] M. Kim, Y. Song, S. Wang, Y. Xia, and X. Jiang, "Secure logistic regression based on homomorphic encryption: Design and evaluation," *JMIR Medical Informatics*, vol. 6, no. 2, p. e19, 2018.

[45] A. Kim, Y. Song, M. Kim, K. Lee, and J. H. Cheon, "Logistic regression model training based on the approximate homomorphic encryption," *BMC Medical Genomics*, vol. 11, no. Suppl 4, p. 83, 2018.

[46] J. Chiang, "Privacy-preserving CNN training with transfer learning: Two hidden layers," *arXiv preprint arXiv:2504.12623*, 2025.

[47] A. M. I. M. Osmani, T. Rahman, M. S. Rahman, and S. Islam, "Priv-FedTL: Privacy-preserving federated transfer learning for resource constrained devices," *Computer Networks*, vol. 286, p. 112449, 2026.

[48] M. Al Amin, M. Hasan, S. Hong, and S. Ullah, "Privacy-preserving federated vision transformer learning with lightweight homomorphic encryption in medical AI," *arXiv preprint arXiv:2511.20983*, 2025.

[49] Y. Li, W. Yu, and J. Zhao, "PrivTuner with homomorphic encryption and loRA: A P3EFT scheme for privacy-preserving parameter-efficient fine-tuning of AI foundation models," *arXiv preprint arXiv:2410.00433*, 2024.

[50] J. Frery, R. Bredehoft, J. Klemsa, A. Meyre, and A. Stoian, "Private loRA fine-tuning of open-source LLMs with homomorphic encryption," *arXiv preprint arXiv:2505.07329*, 2025.

[51] N. Guha, A. Talwalkar, and V. Smith, "One-shot federated learning," *arXiv preprint arXiv:1902.11175*, 2019. [Online]. Available: https://arxiv.org/abs/1902.11175

[52] J. Wang *et al.*, "Towards one-shot federated learning: Advances, challenges, and future directions," *arXiv preprint arXiv:2505.02426*, 2025.

[53] D. Li and J. Wang, "FedMD: Heterogenous federated learning via model distillation," *arXiv preprint arXiv:1910.03581*, 2019. [Online]. Available: https://arxiv.org/abs/1910.03581

[54] S. Itahara, T. Nishio, Y. Koda, M. Morikura, and K. Yamamoto, "DS-FL: Distillation-based semi-supervised federated learning for medical image classification," in *IEEE ICC*, 2021.

[55] T. Lin, L. Kong, S. U. Stich, and M. Jaggi, "Ensemble distillation for robust model fusion in federated learning," in *NeurIPS*, vol. 33, 2020. [Online]. Available: https://proceedings.neurips.cc/paper/2020/hash/18df51b97ccd68128e994804f3eccc87-Abstract.html

[56] J. Zhang, C. Chen, B. Li, L. Lyu, S. Wu, S. Ding, C. Shen, and C. Qi, "DENSE: Data-free one-shot federated learning," in *NeurIPS*, vol. 35, 2022, pp. 21 414–21 428. [Online]. Available: https://arxiv.org/abs/2112.12371

[57] R. Dai, Y. Zhang, A. Li, T. Liu, X. Yang, and B. Han, "Enhancing one-shot federated learning through data and ensemble co-boosting," in *ICLR*, 2024.

[58] Z. Tang, Y. Zhang, P. Dong, Y.-m. Cheung, A. C. Zhou, B. Han, and X. Chu, "Fusefl: One-shot federated learning through the lens of causality with progressive model fusion," in *NeurIPS*, 2024.

[59] X. Liu, L. Liu, F. Ye, Y. Shen, X. Li, L. Jiang, and J. Li, "FedLPA: One-shot federated learning with layer-wise posterior aggregation," in *NeurIPS*, 2024.

[60] Y. Zhang *et al.*, "Federated one-shot learning with data-free distillation," in *NeurIPS*, 2024.

[61] H. Hoech, R. Rischke, K. Mueller, and W. Samek, "FedAUXfdp: Differentially private one-shot federated distillation," in *IJCAI Workshop on Federated Learning*, 2022.

[62] M. Mendieta, G. Sun, and C. Chen, "Navigating heterogeneity and privacy in one-shot federated learning with diffusion models," in *WACV*, 2025, pp. 2601–2610. [Online]. Available: https://openaccess.thecvf.com/content/WACV2025/html/Mendieta_Navigating_Heterogeneity_and_Privacy_in_One-Shot_Federated_Learning_with_Diffusion_Models_WACV_2025_paper.html

[63] Q. Li, B. He, and D. Song, "Practical one-shot federated learning for cross-silo setting," in *IJCAI*, 2021, pp. 1484–1490.

[64] L. Sun and L. Lyu, "Federated model distillation with noise-free differential privacy," in *IJCAI*, 2021, pp. 1563–1569.

[65] F. Tramèr and D. Boneh, "Differentially private learning needs better features (or much more data)," in *International Conference on Learning Representations (ICLR)*, 2021, arXiv:2011.11660.

[66] H. Mehta, W. Krichene, A. Thakurta, A. Kurakin, and A. Cutkosky, "Differentially private image classification from features," *Transactions on Machine Learning Research (TMLR)*, 2023, arXiv:2211.13403.

[67] D. Yu, S. Naik, A. Backurs, S. Gopi, H. A. Inan, G. Kamath, J. Kulkarni, Y. T. Lee, A. Manoel, L. Wutschitz *et al.*, "Differentially private fine-tuning of language models," in *International Conference on Learning Representations (ICLR)*, 2022, arXiv:2110.06500.

[68] X. Li, F. Tramèr, P. Liang, and T. Hashimoto, "Large language models can be strong differentially private learners," in *International Conference on Learning Representations (ICLR)*, 2022, arXiv:2110.05679.

[69] X.-Y. Liu, R. Zhu, D. Zha, J. Gao, S. Zhong, M. White, and M. Qiu, "Differentially private low-rank adaptation of large language model using federated learning," *arXiv preprint arXiv:2312.17493*, 2023.

[70] J. Xu, K. Saravanan, R. van Dalen, H. Mehmood, D. Tuckey, and M. Ozay, "DP-DyLoRA: Fine-tuning transformer-based models on-device under differentially private federated learning using dynamic low-rank adaptation," *arXiv preprint arXiv:2405.06368*, 2024.

[71] J. Zhang, S. Vahidian, M. Kuo, C. Li, R. Zhang, T. Yu, G. Wang, and Y. Chen, "Towards building the federated GPT: Federated instruction tuning," in *IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, 2024, arXiv:2305.05644 (FedIT).

[72] Z. Wang, Z. Shen, Y. He, G. Sun, H. Wang, L. Lyu, and A. Li, "FLoRA: Federated fine-tuning large language models with heterogeneous low-rank adaptations," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2024, arXiv:2409.05976.

[73] J. Bai, D. Chen, B. Qian, L. Yao, and Y. Li, "Federated fine-tuning of large language models under heterogeneous tasks and client resources," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2024, arXiv:2402.11505 (FlexLoRA).

[74] Y. J. Cho, L. Liu, Z. Xu, A. Fahrezi, and G. Joshi, "Heterogeneous LoRA for federated fine-tuning of on-device foundation models," in *Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2024, arXiv:2401.06432.

[75] P. Guo, S. Zeng, Y. Wang, H. Fan, F. Wang, and L. Qu, "Selective aggregation for low-rank adaptation in federated learning," in *International Conference on Learning Representations (ICLR)*, 2025, arXiv:2410.01463 (FedSA-LoRA).

[76] Y. Sun, Z. Li, Y. Li, and B. Ding, "Improving LoRA in privacy-preserving federated learning," in *International Conference on Learning Representations (ICLR)*, 2024, arXiv:2403.12313.

[77] G. Ilharco, M. T. Ribeiro, M. Wortsman, L. Schmidt, H. Hajishirzi, and A. Farhadi, "Editing models with task arithmetic," in *International Conference on Learning Representations (ICLR)*, 2023.

[78] Z. Tao, I. Mason, S. Kulkarni, and X. Boix, "Task arithmetic through the lens of one-shot federated learning," *Transactions on Machine Learning Research (TMLR)*, 2025, arXiv:2411.18607.

[79] Anonymous, "Revisiting federated fine-tuning: A single communication round is enough for foundation models," *arXiv preprint arXiv:2412.04650*, 2024.

[80] J. Zhang, J. Liu, X. Yang, Y. Wang, K. Chen, X. Hou, K. Ren, and X. Yang, "Secure transformer inference made non-interactive," in *Network and Distributed System Security Symposium (NDSS)*, 2025, cryptology ePrint Archive, Paper 2024/136.

[81] M. Hao, H. Li, H. Chen, P. Xing, G. Xu, and T. Zhang, "Iron: Private inference on transformers," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2022, pp. 15 718–15 731.

[82] Q. Pang, J. Zhu, H. Möllering, W. Zheng, and T. Schneider, "BOLT: Privacy-preserving, accurate and efficient inference for transformers," in *IEEE Symposium on Security and Privacy (S&P)*, 2024, pp. 4753–4771.

[83] Y. Dong, W.-j. Lu, Y. Zheng, H. Wu, D. Zhao, J. Tan, Z. Huang, C. Hong, T. Wei, and W. Chen, "PUMA: Secure inference of LLaMA-7B in five minutes," *arXiv preprint arXiv:2307.12533*, 2023.

[84] O. Fontenla-Romero, B. Guijarro-Berdiñas, E. Hernández-Pereira, and B. Pérez-Sánchez, "FedHEONN: Federated and homomorphically encrypted learning method for one-layer neural networks," *Future Generation Computer Systems*, vol. 149, pp. 200–211, 2023.

[85] J. H. Cheon, A. Kim, M. Kim, and Y. Song, "Homomorphic encryption for arithmetic of approximate numbers," in *ASIACRYPT*, 2017, pp. 409–437.

[86] V. Lyubashevsky, C. Peikert, and O. Regev, "On ideal lattices and learning with errors over rings," *Journal of the ACM*, vol. 60, no. 6, pp. 43:1–43:35, 2013.

[87] J. H. Cheon, K. Han, A. Kim, M. Kim, and Y. Song, "Bootstrapping for approximate homomorphic encryption," in *Advances in Cryptology – EUROCRYPT 2018*, ser. Lecture Notes in Computer Science, vol. 10820. Springer, 2018, pp. 360–384. [Online]. Available: https://eprint.iacr.org/2018/153

[88] C. Mouchet, J. R. Troncoso-Pastoriza, J.-P. Bossuat, and J.-P. Hubaux, "Multiparty homomorphic encryption from ring-learning-with-errors," *PoPETs*, vol. 2021, no. 4, pp. 291–311, 2021.

[89] F. Tramèr, F. Zhang, A. Juels, M. K. Reiter, and T. Ristenpart, "Stealing machine learning models via prediction APIs," in *25th USENIX Security Symposium (USENIX Security 16)*, 2016, pp. 601–618.

[90] N. Carlini, D. Paleka, K. D. Dvijotham, T. Steinke *et al.*, "Stealing part of a production language model," in *Proceedings of the 41st International Conference on Machine Learning (ICML)*, 2024, arXiv:2403.06634.

[91] Y. Chen, X. Dong, J. Guo, Y. Shen, A. Wang, and X. Wang, "Hard-label cryptanalytic extraction of neural network models," in *Advances in Cryptology – ASIACRYPT 2024*. Springer, 2024.

[92] N. Carlini, J. Chávez-Saab, A. Hambitzer, F. Rodríguez-Henríquez, and A. Shamir, "Polynomial time cryptanalytic extraction of deep neural networks in the hard-label setting," in *Advances in Cryptology – EUROCRYPT 2025*. Springer, 2025.

[93] J. H. Cheon, D. Kim, and D. Kim, "Efficient homomorphic comparison methods with optimal complexity," in *Advances in Cryptology – ASIACRYPT 2020*, ser. Lecture Notes in Computer Science, vol. 12492, 2020, pp. 221–256.

[94] A. Viand, C. Knabenhans, and A. Hithnawi, "SoK: Fully homomorphic encryption compilers and verifiable computation," in *IEEE S&P*, 2023. [Online]. Available: https://arxiv.org/abs/2301.07041

[95] T.-M. H. Hsu, H. Qi, and M. Brown, "Measuring the effects of non-identical data distribution for federated visual classification," *arXiv preprint arXiv:1909.06335*, 2019.

[96] Q. Li, Y. Diao, Q. Chen, and B. He, "Federated learning on non-IID data silos: An experimental study," in *2022 IEEE 38th International Conference on Data Engineering (ICDE)*. IEEE, 2022, pp. 965–978.

[97] C. Mouchet, J.-P. Bossuat, J. Troncoso-Pastoriza, and J.-P. Hubaux, "Lattigo v5: A multiparty homomorphic encryption library in Go," in *WAHC*, 2021.

[98] F. McSherry and K. Talwar, "Mechanism design via differential privacy," in *48th Annual IEEE Symposium on Foundations of Computer Science (FOCS)*, 2007, pp. 94–103.

HALİL İBRAHİM KANPAK received his B.Sc. degree in Mathematics from Koç University. He is currently pursuing his Ph.D. at Koç University, where he is a member of the Cryptography, Cyber Security & Privacy Research Group. His research interests include cryptography and related areas in Deep Learning.

Asst. Prof. SİNEM SAV received her Ph.D. in Computer and Communication Sciences from École Polytechnique Fédérale de Lausanne (EPFL). She also holds M.Sc. and B.Sc. degrees in Computer Engineering from Bilkent University, where she currently serves as an Assistant Professor in the Computer Engineering department. Her research interests include privacy-preserving machine learning and applied cryptography.

Prof. ALPTEKİN KÜPÇÜ received his Ph.D. degree from Brown University Computer Science Department in 2010. Since then, he has been working as a faculty member at Koç University, leading the Cryptography, Cyber Security & Privacy Research Group that he founded. He is an ACM and IEEE Senior Member, and a co-founder of Xtinge Technology Inc.